

ΓΡΑΦΙΚΑ ΥΠΟΛΟΓΙΣΤΩΝ ΚΑΙ
ΣΥΣΤΗΜΑΤΑ ΑΛΛΗΛΕΠΙΔΡΑΣΗΣ

1Γ ΠΡΟΓΡΑΜΜΑΤΙΣΤΙΚΗ ΕΡΓΑΣΙΑ
ΓΙΑ ΤΟ ΑΚΑΔΗΜΑΪΚΟ ΕΤΟΣ 2024-
2025

ΟΜΑΔΑ 5380-5226

ΙΩΑΝΝΗΣ ΦΙΛΛΗΣ, ΑΜ:5380

ΚΩΝΣΤΑΝΤΙΝΟΣ ΖΩΗΣ, ΑΜ:5226

ΤΕΛΙΚΗ ΑΝΑΦΟΡΑ

1 ΕΙΣΑΓΩΓΗ

Η παρούσα αναφορά συγγράφεται στα πλαίσια της τρίτης εργαστηριακής άσκησης/προγραμματιστικής άσκησης (1Γ) του μαθήματος Γραφικά Υπολογιστών και Συστήματα Αλληλεπίδρασης από τους φοιτητές Γιάννη Φίλλη (ΑΜ: 5380) και Κωνσταντίνο Ζώη (ΑΜ:5226).

Σκοπός αυτής της άσκησης είναι να εξασκηθούμε στην χρήση των βασικών βιβλιοθηκών στοιχειωδών γραφικών της OpenGL. Ειδικότερα, στην συγκεκριμένη άσκηση έπρεπε να δημιουργήσουμε μία εφαρμογή-παιχνίδι στο οποίο ένας χαρακτήρας θα κυνηγάει θησαυρούς.

2 ΕΡΩΤΗΜΑ 1: ΑΝΟΙΓΜΑ ΠΑΡΑΘΥΡΟΥ ΚΑΙ ΤΙΤΛΟΣ

Στο πρώτο ερώτημα, όπως και στην προηγούμενη άσκηση, αυτό που καλούμαστε να κάνουμε, είναι αρχικά να δημιουργήσουμε το παράθυρο με διαστάσεις 950x950, να θέσουμε το background του παραθύρου σε χρώμα μαύρο, να θέσουμε τον τίτλο του παραθύρου **Άσκηση 1B-2024** και η εφαρμογή να τερματίζει με το πλήκτρο **SPACE**.

Έτσι λοιπόν, όπως φαίνεται και παρακάτω, πάμε στην εντολή **glfwCreateWindow()** και θέτουμε τις δύο παραμέτρους για το μέγεθος 950 και το όνομα στο επιθυμητό, ωστόσο θα πρέπει στην αρχή να βάλουμε το u8 ώστε να μπορεί να αναγνωρίσει τους ελληνικούς χαρακτήρες.

```
window = glfwCreateWindow(950, 950, u8"Άσκηση 1B - 2024- Κυνήγι Θυσσαυρού", NULL, NULL);
```

Εικόνα 2.1: Εντολή δημιουργίας παραθύρου

Έπειτα για να αλλάξουμε το χρώμα του background σε μαύρο χρησιμοποιούμε την εντολή **glClearColor()** και με παραμέτρους 0.0f, 0.0f, 0.0f θέτουμε το χρώμα σε μαύρο (σε rgb).

```
//black background  
glClearColor(0.0f, 0.0f, 0.0f, 0.0f);
```

Εικόνα 2.2: Εντολή αλλαγής χρώματος σε μαύρο

Για το τελευταίο ζητούμενο αυτού του ερωτήματος, θα βρούμε στον κώδικα το κύριο loop και θα τροποποιήσουμε την συνθήκη του, ώστε ο έλεγχος για το αν θα κλείσει το παράθυρο να γίνεται για το κουμπί SPACE.

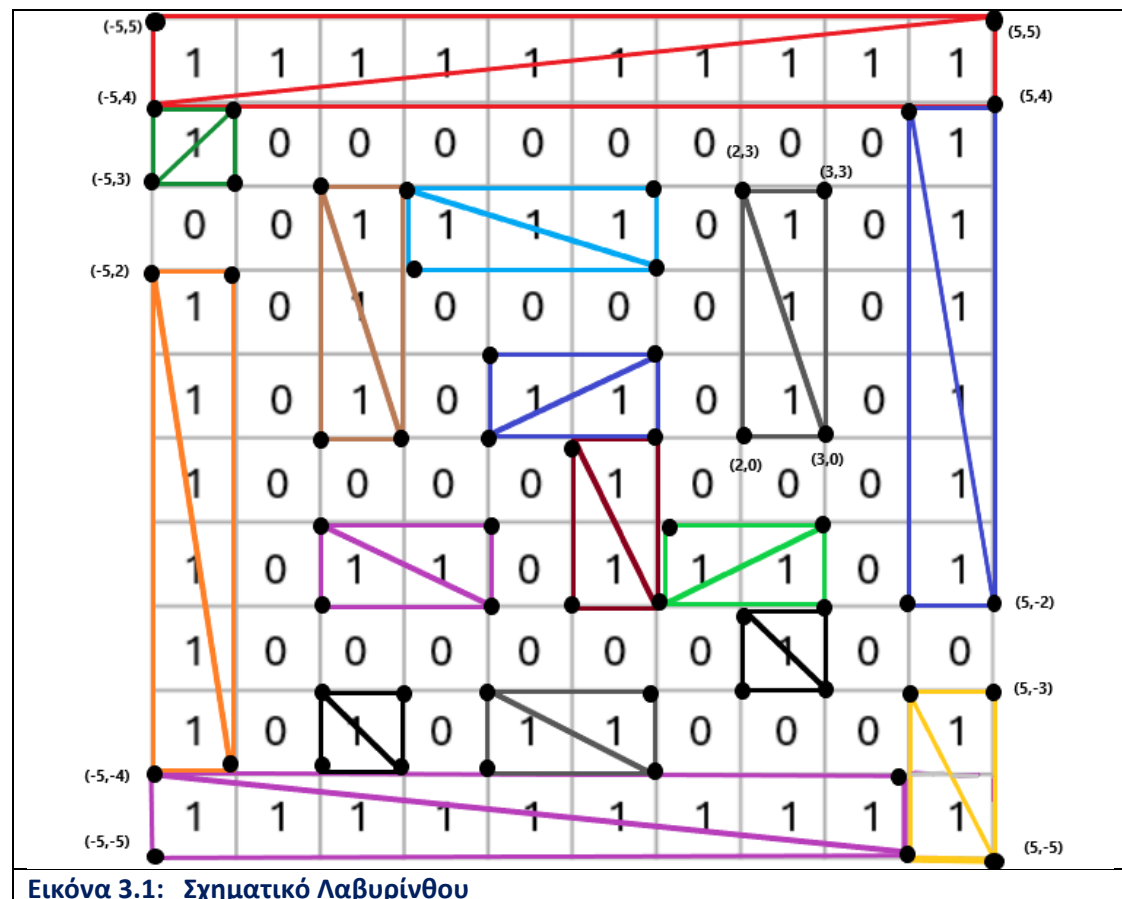
```
} // Check if the ESC key was pressed or the window was closed  
while (glfwGetKey(window, GLFW_KEY_SPACE) != GLFW_PRESS &&  
glfwWindowShouldClose(window) == 0);
```

Εικόνα 2.3: Εντολή αλλαγής χρώματος σε μαύρο

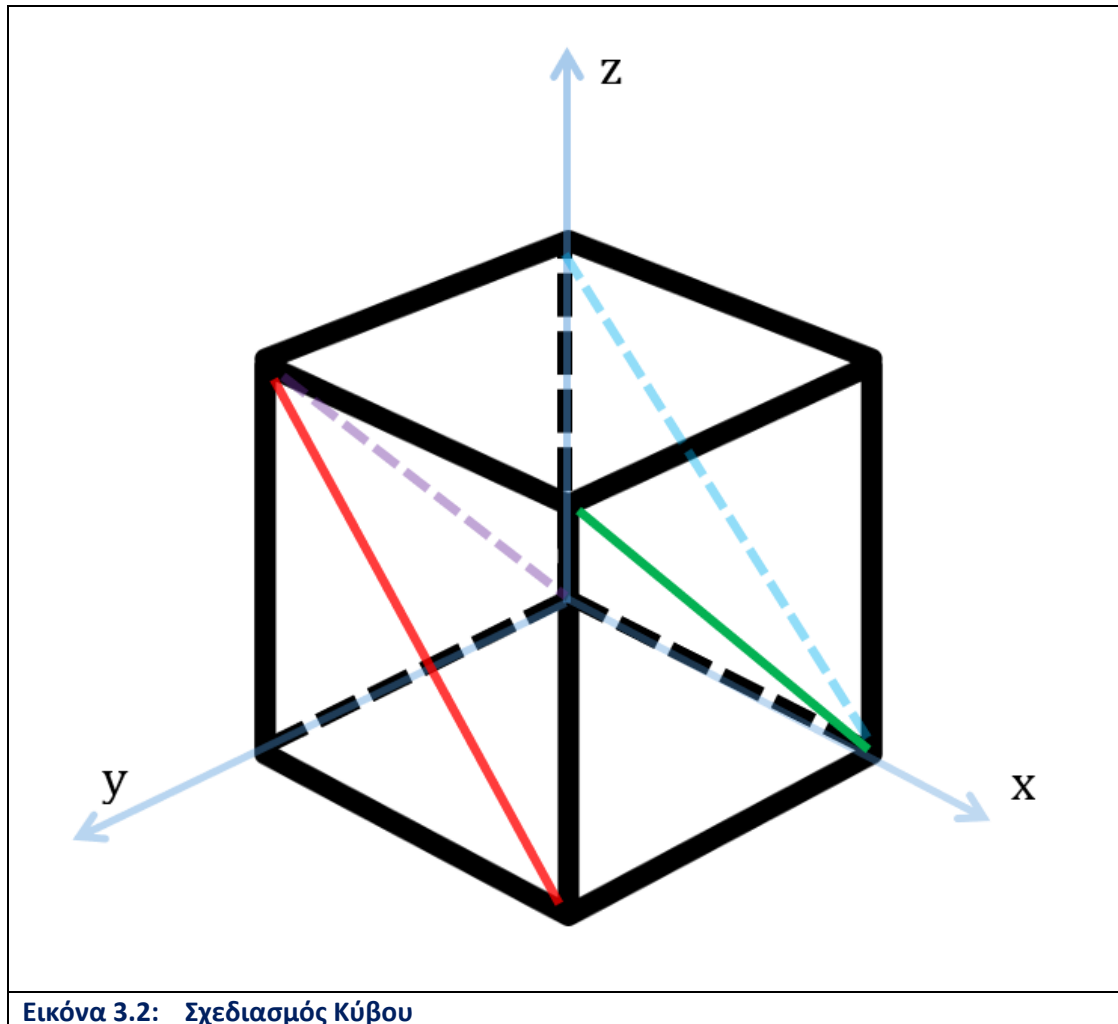
3 ΕΡΩΤΗΜΑ II

3.1 Α: ΛΑΒΥΡΙΝΘΟΣ

Για την σχεδίαση του λαβυρίνθου, η υλοποίηση είναι ακριβώς ίδια με την προηγούμενη άσκηση. Χωρίσαμε τους τοίχους όπως και στην πρώτη άσκηση, με τα τρίγωνα που φαίνονται στην εικόνα 3.1. Θεωρούμε πως αυτό που βλέπουμε στην εικόνα είναι οι κάτω πλευρές των τοίχων, ότι ακουμπάνε δηλαδή στο επίπεδο xy . Τώρα πρέπει να μετατρέψουμε το κάθε ορθογώνιο ή τετράγωνο του λαβυρίνθου σε σχήμα τριών διαστάσεων.

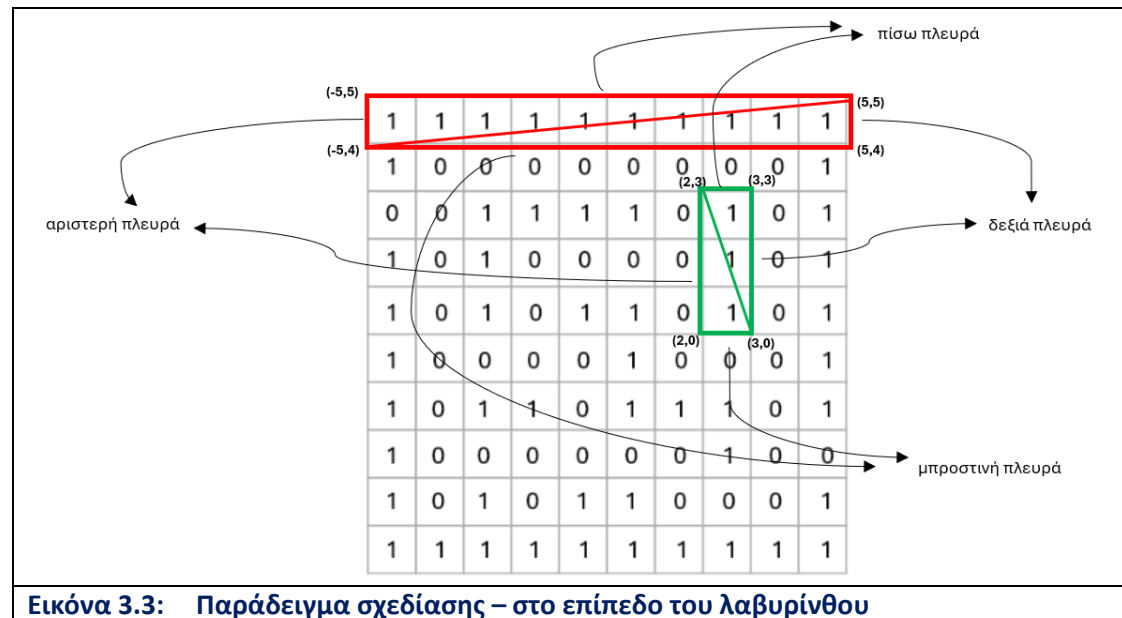


Γνωρίζουμε ότι για να φτιάξουμε έναν κύβο σε 3 διαστάσεις στην OpenGL πρέπει να σχεδιάσουμε και τις 6 πλευρές του και να χωρίσουμε την κάθε πλευρά σε 2 τρίγωνα. Άρα χρειαζόμαστε 12 τρίγωνα για κάθε σχήμα που θα φτιάξουμε. Θεωρούμε λοιπόν έναν στοιχειώδες κύβο όπως αυτόν που φαίνεται στην εικόνα 3.2. Αυτό το σχήμα χρησιμοποιήσαμε σαν οδηγό για κάθε διαφορετικό block του λαβυρίνθου.



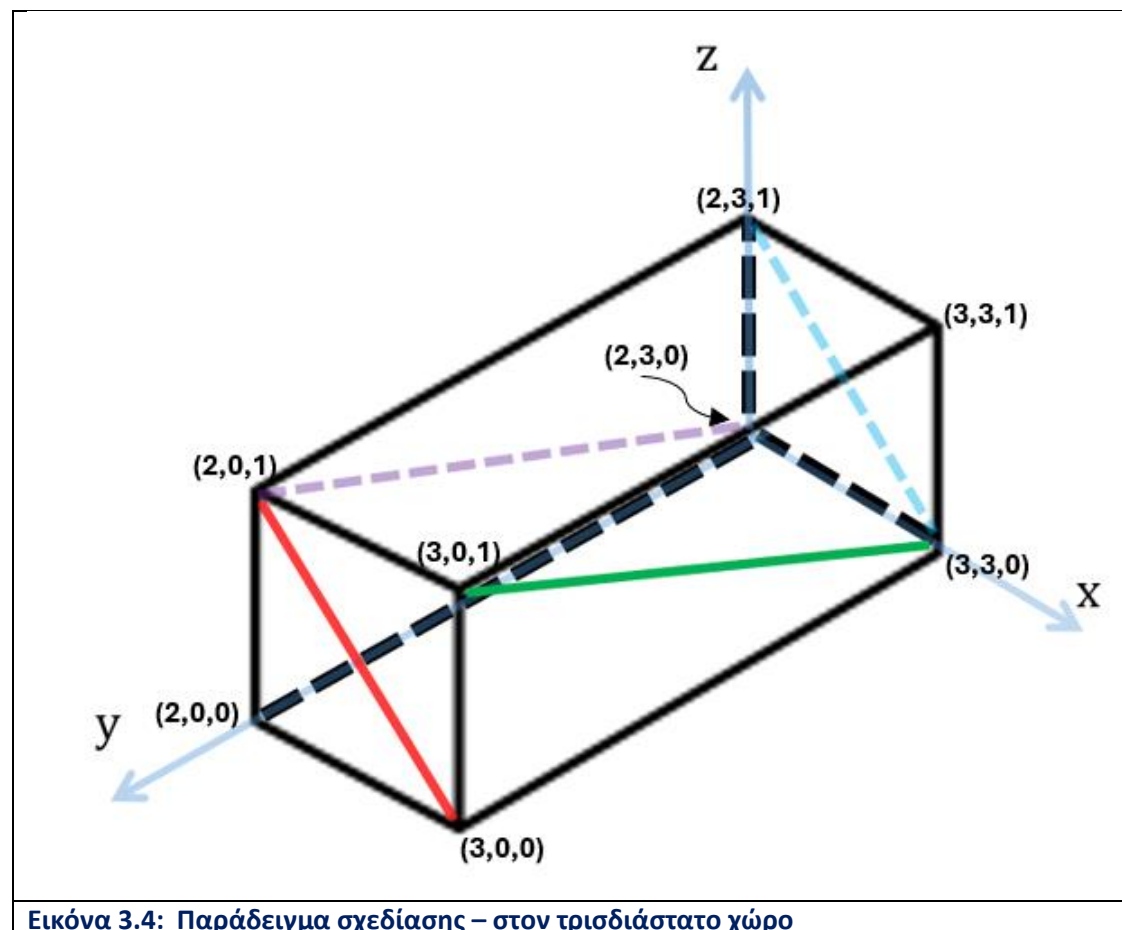
Σε αυτή την εικόνα βλέπουμε πως έχουμε χωρίσει τα τρίγωνα για την κάθε πλευρά του κύβου. Η μπροστινή πλευρά χωρίζεται από την κόκκινη τομή, η δεξιά πλευρά από την πράσινη, η αριστερή από την μωβ και η πίσω πλευρά από την γαλάζια. Η κάτω και η πάνω πλευρές χωρίζονται με τον ίδιο τρόπο και ανάλογα το block όπως στην εικόνα 3.1 όπου θεωρούμε ότι κοιτάμε τον λαβύρινθο από πάνω. Η πάνω πλευρά είναι ουσιαστικά ίδια με την κάτω απλά διαφέρει κατά 1 στα σημεία του άξονα ζ, βρίσκεται δηλαδή μία μονάδα ψηλότερα.

Όπως αναφέραμε, κάθε block του λαβυρίνθου χωρίζεται με τον ίδιο τρόπο που εξηγήσαμε παραπάνω. Ας δούμε ένα παράδειγμα:



Εικόνα 3.3: Παράδειγμα σχεδίασης – στο επίπεδο του λαβυρίνθου

Στην εικόνα 3.3 βλέπουμε πως αντιλαμβανόμαστε την κάθε πλευρά των σχημάτων στο επίπεδο του λαβυρίνθου. Ας δούμε τώρα συγκεκριμένα για το πράσινο ορθογώνιο, την μεταφορά του σε τρισδιάστατο χώρο και τον ορισμό των σημείων του. Οι τομές των πλευρών βασίζονται ξανά στον κύβο-οδηγό της εικόνας 3.2.



Εικόνα 3.4: Παράδειγμα σχεδίασης – στον τρισδιάστατο χώρο

Τώρα ας δούμε τον ορισμό των vertices αυτού του σχήματος στην OpenGL μέσα στον πίνακα *maze_buffer*.

836	// (2,3) (3,3) (2,0) (3,0) block	858	//back
837	//bottom	859	2.0f, 3.0f, 0.0f,
838	2.0f, 3.0f, 0.0f,	860	2.0f, 3.0f, 1.0f,
839	2.0f, 0.0f, 0.0f,	861	3.0f, 3.0f, 0.0f,
840	3.0f, 0.0f, 0.0f,	862	3.0f, 3.0f, 0.0f,
841	2.0f, 3.0f, 0.0f,	863	3.0f, 3.0f, 1.0f,
842	3.0f, 3.0f, 0.0f,	864	2.0f, 3.0f, 1.0f,
843	3.0f, 0.0f, 0.0f,	865	//right
844	//up	866	3.0f, 0.0f, 0.0f,
845	2.0f, 3.0f, 1.0f,	867	3.0f, 0.0f, 1.0f,
846	2.0f, 0.0f, 1.0f,	868	3.0f, 3.0f, 0.0f,
847	3.0f, 0.0f, 1.0f,	869	3.0f, 3.0f, 0.0f,
848	2.0f, 3.0f, 1.0f,	870	3.0f, 3.0f, 1.0f,
849	3.0f, 3.0f, 1.0f,	871	3.0f, 0.0f, 1.0f,
850	3.0f, 0.0f, 1.0f,	872	//left
851	//front	873	2.0f, 0.0f, 0.0f,
852	2.0f, 0.0f, 0.0f,	874	2.0f, 0.0f, 1.0f,
853	2.0f, 0.0f, 1.0f,	875	2.0f, 3.0f, 0.0f,
854	3.0f, 0.0f, 0.0f,	876	2.0f, 3.0f, 0.0f,
855	3.0f, 0.0f, 0.0f,	877	2.0f, 3.0f, 1.0f,
856	3.0f, 0.0f, 1.0f,	878	2.0f, 0.0f, 1.0f,
857	2.0f, 0.0f, 1.0f,		

Εικόνα 3.5: Παράδειγμα σχεδίασης – αναπαράσταση στην OpenGL

Παρόμοια ορίζουμε και τα υπόλοιπα block του λαβυρίνθου αποθηκεύοντας τα vertices στον πίνακα *maze_buffer*. Για κάθε σχήμα έχουμε 8 μοναδικά σημεία, και 3 σημεία για κάθε τρίγωνο, η κάθε πλευρά έχει 2 τρίγωνα, άρα σχεδιάζουμε 36 σημεία. Παρατηρούμε επίσης ότι η πάνω πλευρά είναι ίδια με την κάτω αλλά με τη z συντεταγμένη ίση με 1, η πίσω πλευρά είναι ίδια με την μπροστινή αλλά με διαφορετικό y και η αριστερή πλευρά είναι ίδια με την δεξιά αλλά με διαφορετικό x.

Αφού ολοκληρώσουμε τον ορισμό των συντεταγμένων στον πίνακα, πρέπει να δημιουργήσουμε τα VBO για τον σχεδιασμό του λαβυρίνθου και για το χρώμα του.

1879	GLuint vertexbuffer;
1880	glGenBuffers(1, &vertexbuffer);
1881	glBindBuffer(GL_ARRAY_BUFFER, vertexbuffer);
1882	glBufferData(GL_ARRAY_BUFFER, sizeof(maze), maze, GL_STATIC_DRAW);
1883	
1884	GLuint colorbuffer;
1885	glGenBuffers(1, &colorbuffer);
1886	glBindBuffer(GL_ARRAY_BUFFER, colorbuffer);
1887	glBufferData(GL_ARRAY_BUFFER, sizeof(color), color, GL_STATIC_DRAW);
1888	

Εικόνα 3.6: Ορισμός των VBO

Στη συνέχεια χρειαζόμαστε έναν πίνακα για τα χρώματα ώστε να τον κάνουμε bind μετά με τον *colorbuffer*. Για το μπλε χρώμα η τρίτη μεταβλητή πρέπει να είναι ίση με 1 αφού δουλεύουμε σε τύπο RGB. Το μέγεθος του πίνακα *color* είναι ίσο με αυτό του λαβυρίνθου

επειδή πρέπει να περαστεί κάθε μπλοκ του **color** σε κάθε σχήμα του λαβυρίνθου. Επειδή έχουμε χωρίσει τον λαβύρινθο σε 16 σχήματα, 16 θα είναι και τα μπλοκ του χρώματος. Ένα παράδειγμα του πίνακα φαίνεται παρακάτω.

```
1189
1190     GLfloat a = 0.4f;
1191     static const GLfloat color[] = {
1192         0.0f, 0.0f, 1.0f, a,
1193         0.0f, 0.0f, 1.0f, a,
1194         0.0f, 0.0f, 1.0f, a,
1195         0.0f, 0.0f, 1.0f, a,
1196         0.0f, 0.0f, 1.0f, a,
1197         0.0f, 0.0f, 1.0f, a,
1198         0.0f, 0.0f, 1.0f, a,
1199         0.0f, 0.0f, 1.0f, a,
1200         0.0f, 0.0f, 1.0f, a,
1201         0.0f, 0.0f, 1.0f, a,
1202         0.0f, 0.0f, 1.0f, a,
1203         0.0f, 0.0f, 1.0f, a,
1204         0.0f, 0.0f, 1.0f, a,
1205         0.0f, 0.0f, 1.0f, a,
1206         0.0f, 0.0f, 1.0f, a,
1207         0.0f, 0.0f, 1.0f, a,
1208         0.0f, 0.0f, 1.0f, a,
1209         0.0f, 0.0f, 1.0f, a,
1210         0.0f, 0.0f, 1.0f, a,
1211         0.0f, 0.0f, 1.0f, a,
1212         0.0f, 0.0f, 1.0f, a,
1213         0.0f, 0.0f, 1.0f, a,
1214         0.0f, 0.0f, 1.0f, a,
1215         0.0f, 0.0f, 1.0f, a,
1216         0.0f, 0.0f, 1.0f, a,
1217         0.0f, 0.0f, 1.0f, a,
1218         0.0f, 0.0f, 1.0f, a,
1219         0.0f, 0.0f, 1.0f, a,
1220         0.0f, 0.0f, 1.0f, a,
1221         0.0f, 0.0f, 1.0f, a,
1222         0.0f, 0.0f, 1.0f, a,
1223         0.0f, 0.0f, 1.0f, a,
1224         0.0f, 0.0f, 1.0f, a,
1225         0.0f, 0.0f, 1.0f, a,
1226         0.0f, 0.0f, 1.0f, a,
1227         0.0f, 0.0f, 1.0f, a,
1228
```

Εικόνα 3.7: Το πρώτο μπλοκ του πίνακα color

Τέλος, μας μένει να σχεδιάσουμε τον λαβύρινθο κάνοντας bind τα σημεία του (attribute 0) και χρωματίζοντας τον (attribute 1). Στην **glDrawArrays()** κάνουμε 16*36 καθώς όπως είπαμε ο λαβύρινθος αποτελείται από 16 σχήματα με 36 σημεία το καθένα. Για το **mazenormalbuffer** καθώς και για το “useTexture” θα αναφερθούμε παρακάτω.


```
// maze
glUniform1i(glGetUniformLocation(programID, "useTexture"), GL_FALSE);
glEnableVertexAttribArray(0);
glBindBuffer(GL_ARRAY_BUFFER, vertexbuffer);
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 0, (void*)0);

// color maze
glEnableVertexAttribArray(1);
glBindBuffer(GL_ARRAY_BUFFER, colorbuffer);
glVertexAttribPointer(1, 4, GL_FLOAT, GL_FALSE, 0, (void*)0);

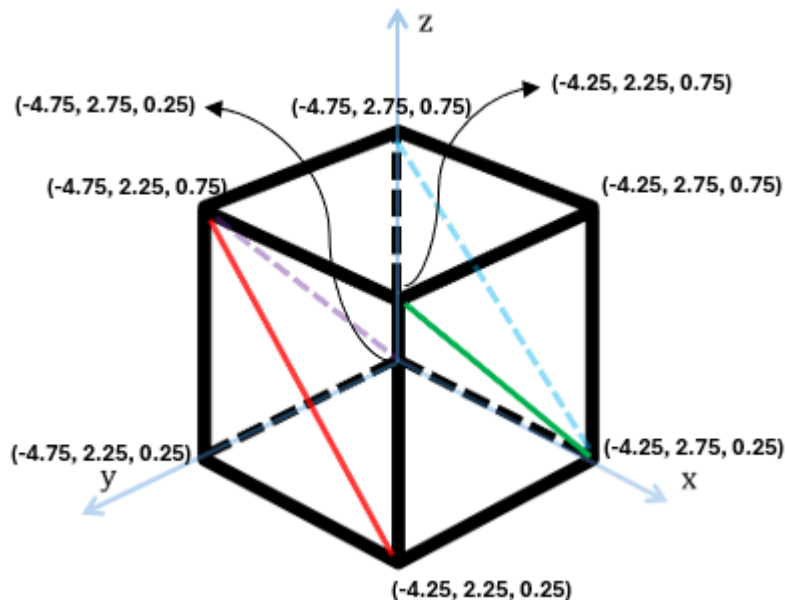
//maze normals
glEnableVertexAttribArray(3);
glBindBuffer(GL_ARRAY_BUFFER, mazenormalbuffer);
glVertexAttribPointer(3, 3, GL_FLOAT, GL_FALSE, 0, (void*)0);

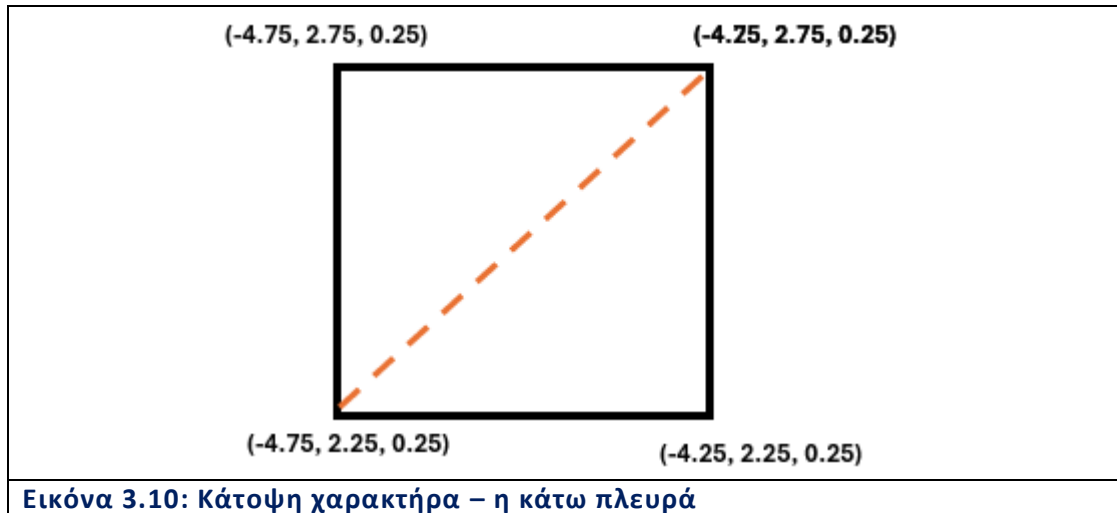
// Draw maze
glDrawArrays(GL_TRIANGLES, 0, 16 * 36);
```

Εικόνα 3.8: Σχεδιασμός και χρωματισμός λαβυρίνθου

3.2 B: ΧΑΡΑΚΤΗΡΑΣ

Ο χαρακτήρας σχεδιάζεται με παρόμοιο τρόπο όπως και κάθε σχήμα του λαβυρίνθου. Ο προσδιορισμός των συντεταγμένων του είναι ίδιος με την πρώτη άσκηση. Όμως επειδή τώρα δουλεύουμε σε 3 διαστάσεις και ο χαρακτήρας βρίσκεται στο κέντρο, ορίσαμε τις z συντεταγμένες του ως 0.75 για την πάνω πλευρά και 0.25 για την κάτω. Με αυτό τον τρόπο ο χαρακτήρας είναι ακριβώς στη μέση συγκριτικά με τους τοίχους και διατηρεί σχήμα κύβου. Πιο συγκεκριμένα, ο προσδιορισμός των πλευρών και ο διαχωρισμός των τριγώνων του είναι ίδιος με το υπόδειγμα που θεωρήσαμε αρχικά στην εικόνα 3.2.

**Εικόνα 3.9: Προσδιορισμός τριγώνων και συντεταγμένων χαρακτήρα**



Εικόνα 3.10: Κάτοψη χαρακτήρα – η κάτω πλευρά

Έτσι, ορίζουμε όλα τα σημεία στον πίνακα *character_buffer*.

```
1784
1785 static GLfloat character_buffer[] = {
1786     // (-4.75,2.25) (-4.25,2.25) (-4.75,2.75) (-4.25,2.75) block
1787     //bottom
1788     -4.75f, 2.75f, 0.25f, //xmin,ymax point
1789     -4.75f, 2.25f, 0.25f,
1790     -4.25f, 2.75f, 0.25f,
1791     -4.75f, 2.25f, 0.25f,
1792     -4.25f, 2.75f, 0.25f,
1793     -4.25f, 2.25f, 0.25f,
1794     //up
1795     -4.75f, 2.75f, 0.75f,
1796     -4.75f, 2.25f, 0.75f,
1797     -4.25f, 2.75f, 0.75f,
1798     -4.75f, 2.25f, 0.75f,
1799     -4.25f, 2.75f, 0.75f,
1800     -4.25f, 2.25f, 0.75f,
1801     //front
1802     -4.75f, 2.25f, 0.25f,
1803     -4.75f, 2.25f, 0.75f,
1804     -4.25f, 2.25f, 0.25f,
1805     -4.25f, 2.25f, 0.25f,
1806     -4.25f, 2.25f, 0.75f,
1807     -4.75f, 2.25f, 0.75f,
1808     //back
1809     -4.75f, 2.75f, 0.25f,
1810     -4.75f, 2.75f, 0.75f,
1811     -4.25f, 2.75f, 0.25f,
1812     -4.25f, 2.75f, 0.25f,
1813     -4.25f, 2.75f, 0.75f,
1814     -4.75f, 2.75f, 0.75f,
1815     //right
1816     -4.25f, 2.25f, 0.25f,
1817     -4.25f, 2.25f, 0.75f,
1818     -4.25f, 2.75f, 0.25f,
1819     -4.25f, 2.75f, 0.25f,
1820     -4.25f, 2.75f, 0.75f,
1821     -4.25f, 2.25f, 0.75f,
1822     //left
1823     -4.75f, 2.25f, 0.25f,
1824     -4.75f, 2.25f, 0.75f,
1825     -4.75f, 2.75f, 0.25f,
1826     -4.75f, 2.75f, 0.25f,
1827     -4.75f, 2.75f, 0.75f,
1828     -4.75f, 2.25f, 0.75f
1829 };
1830
```

Εικόνα 3.11: Ο πίνακας *character_buffer*

Το χρώμα του χαρακτήρα θέλουμε να είναι κίτρινο, άρα θα ορίσουμε άλλον έναν πίνακα **color2** όπου η τιμή του κόκκινου και του πράσινου θα είναι ίση με 1 για να πάρουμε το κίτρινο χρώμα (εικόνα 4.4).

Φυσικά πρέπει να ορίσουμε και τα κατάλληλα VBO για τον χαρακτήρα (**characterbuffer**) και το νέο χρώμα (**colorbuffer2**) όπως φαίνεται στην εικόνα 4.5

```
1831
1832 static const GLfloat color2[] = {
1833     1.0f, 1.0f, 0.0f, a,
1834     1.0f, 1.0f, 0.0f, a,
1835     1.0f, 1.0f, 0.0f, a,
1836     1.0f, 1.0f, 0.0f, a,
1837     1.0f, 1.0f, 0.0f, a,
1838     1.0f, 1.0f, 0.0f, a,
1839     1.0f, 1.0f, 0.0f, a,
1840     1.0f, 1.0f, 0.0f, a,
1841     1.0f, 1.0f, 0.0f, a,
1842     1.0f, 1.0f, 0.0f, a,
1843     1.0f, 1.0f, 0.0f, a,
1844     1.0f, 1.0f, 0.0f, a,
1845     1.0f, 1.0f, 0.0f, a,
1846     1.0f, 1.0f, 0.0f, a,
1847     1.0f, 1.0f, 0.0f, a,
1848     1.0f, 1.0f, 0.0f, a,
1849     1.0f, 1.0f, 0.0f, a,
1850     1.0f, 1.0f, 0.0f, a,
1851     1.0f, 1.0f, 0.0f, a,
1852     1.0f, 1.0f, 0.0f, a,
1853     1.0f, 1.0f, 0.0f, a,
1854     1.0f, 1.0f, 0.0f, a,
1855     1.0f, 1.0f, 0.0f, a,
1856     1.0f, 1.0f, 0.0f, a,
1857     1.0f, 1.0f, 0.0f, a,
1858     1.0f, 1.0f, 0.0f, a,
1859     1.0f, 1.0f, 0.0f, a,
1860     1.0f, 1.0f, 0.0f, a,
1861     1.0f, 1.0f, 0.0f, a,
1862     1.0f, 1.0f, 0.0f, a,
1863     1.0f, 1.0f, 0.0f, a,
1864     1.0f, 1.0f, 0.0f, a,
1865     1.0f, 1.0f, 0.0f, a,
1866     1.0f, 1.0f, 0.0f, a,
1867     1.0f, 1.0f, 0.0f, a,
1868     1.0f, 1.0f, 0.0f, a
1869 };
1870
```

Εικόνα 3.12: Ο πίνακας color2

```
GLuint characterbuffer;  
glGenBuffers(1, &characterbuffer);  
  
GLuint colorbuffer2;  
glGenBuffers(1, &colorbuffer2);  
glBindBuffer(GL_ARRAY_BUFFER, colorbuffer2);  
glBufferData(GL_ARRAY_BUFFER, sizeof(color2), color2, GL_STATIC_DRAW);
```

Εικόνα 3.13: Ορισμός των VBO

Τέλος, για την τελική σχεδίαση του χαρακτήρα κάνουμε bind μέσα στο βρόγχο του πίνακά του ώστε να μπορεί να σχεδιάζεται δυναμικά και να αλλάζει η θέση του σε κάθε επανάληψη με το κατάλληλο input. Κάνουμε bind και το νέο χρώμα στο attribute 1 και είμαστε έτοιμοι να καλέσουμε την *glDrawArrays()* με όρισμα 36 αυτή τη φορά αφού έχουμε μόνο έναν κύβο.

```
1928  
1929 // character  
1930 glEnableVertexAttribArray(0);  
1931 glBindBuffer(GL_ARRAY_BUFFER, characterbuffer);  
1932 glBufferData(GL_ARRAY_BUFFER, sizeof(character_buffer), character_buffer, GL_DYNAMIC_DRAW);  
1933  
1934 glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 0, (void*)0);  
1935  
1936 // color character  
1937 glEnableVertexAttribArray(1);  
1938 glBindBuffer(GL_ARRAY_BUFFER, colorbuffer2);  
1939 glVertexAttribPointer(1, 4, GL_FLOAT, GL_FALSE, 0, (void*)0);  
1940  
1941 //draw character  
1942 glDrawArrays(GL_TRIANGLES, 0, 36);  
1943
```

Εικόνα 3.14: Σχεδιασμός και χρωματισμός χαρακτήρα

Σχετικά με την κίνηση του χαρακτήρα και τη σύγκρουση σε τοίχους, η υλοποίηση είναι ίδια με την δεύτερη άσκηση: Ορίζουμε την αναπαράσταση του χάρτη του λαβυρίνθου με την *createMap()*, με την *checkMove()* μέσω της *upscalePlayer()* ελέγχουμε αν η κίνηση είναι έγκυρη και αν ναι, την πραγματοποιούμε με την συνάρτηση *moveSquare()*. Για τον έλεγχο της σύγκρουσης πάλι παίρνουμε το πρώτο σημείο που ορίσαμε στον πίνακα και ξέρουμε ότι αυτό είναι το (x_{min}, y_{max}) . Αρκεί να γίνει ο έλεγχος μόνο με αυτό το σημείο ξανά καθώς οι πλευρές του κύβου δεν τον επηρεάζουν.

Θέλουμε ο χαρακτήρας να μεταφέρεται στην έξοδο αν κινηθεί αριστερά από την είσοδο και να μεταφέρεται στην είσοδο αν κινηθεί δεξιά από την έξοδο. Για να το καταφέρουμε αυτό, υλοποιήσαμε τις δύο συναρτήσεις *isAtStart()* και *isAtEnd()* οι οποίες ελέγχουν αν το σημείο (x_{min}, y_{max}) βρίσκεται στη θέση της εκκίνησης ή του τερματισμού αντίστοιχα ελέγχοντας επακριβώς τις συντεταγμένες του.

```
213
214  ✓ bool isAtEnd(GLfloat* player) {
215      //check (xmin,ymax) point of exit block
216      ✓ if (player[0] == 4.25f && player[1] == -2.25f) {
217          return true;
218      }
219      ✓ else {
220          return false;
221      }
222  }
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
```

Εικόνα 3.15: Οι συναρτήσεις isAtStart και isAtEnd

Στη συνέχεια, οι συναρτήσεις *moveToStart()* και *moveToEnd()* μεταφέρουν τον χαρακτήρα στην αρχή και αντίστοιχα στο τέλος του λαβυρίνθου. Αλλάζουν όλα τα σημεία του για το x και το y ξεχωριστά. Όλα τα x θα διαφέρουν κατά μια συγκεκριμένη απόσταση και το y το ίδιο. Επομένως, αφού οι θέσεις προς και από την μεταφορά είναι συγκεκριμένες και τις γνωρίζουμε, αρκεί να προσθέσουμε ή να αφαιρέσουμε τη διαφορά τους κατά τις x συντεταγμένες και το ίδιο κατά τις y.

```
202
203  ✓ void moveToStart(GLfloat* player) {
204      //x
205      ✓ for (int i = 0; i < 108; i = i + 3) {
206          player[i] = player[i] - 9.0f;
207      }
208      //y
209      ✓ for (int i = 1; i < 108; i = i + 3) {
210          player[i] = player[i] + 5.0f;
211      }
212  }
213
214
215
216
217
218
219
220
221
222
223
224  ✓ void moveToEnd(GLfloat* player) {
225      //x
226      ✓ for (int i = 0; i < 108; i = i + 3) {
227          player[i] = player[i] + 9.0f;
228      }
229      //y
230      ✓ for (int i = 1; i < 108; i = i + 3) {
231          player[i] = player[i] - 5.0f;
232      }
233  }
234
```

Εικόνα 3.16: Οι συναρτήσεις moveToStart και moveToEnd

Οι συναρτήσεις αυτές καλούνται από την **moveSquare()** κατά την κίνηση του χαρακτήρα. Άρα βάζουμε ελέγχους if μέσα στην συνάρτηση **moveSquare()**. Στην κίνηση δεξιά ελέγχουμε αρχικά με την **isAtEnd()** αν ο χαρακτήρας βρίσκεται στο τέλος και αν ναι καλούμε την **moveToStart()** για την μεταφορά του στην εκκίνηση. Παρόμοια, στην κίνηση αριστερά ελέγχουμε αν ο χαρακτήρας βρίσκεται στην αρχή του λαβυρίνθου (**isAtStart()**) και αν ναι τον μεταφέρουμε στο τέλος (**moveToEnd()**).

```
if (move == 1) { //move right
    //check if player is at exit point
    if (isAtEnd(player)) {
        moveToStart(player);
    }
    else {
        for (int i = 0; i < 100; i = i + 3) {
            temp[i] = temp[i] + 1.0f;
        }
        if (checkMove(map, temp) != false) {
            for (int i = 0; i < 100; i = i + 3) {
                player[i] = player[i] + 1.0f;
            }
        }
    }
}

else if (move == 2) { // move left
    //check if player is at starting point
    if (isAtStart(player)) {
        moveToEnd(player);
    }
    else {
        for (int i = 0; i < 100; i = i + 3) {
            temp[i] = temp[i] - 1.0f;
        }
        if (checkMove(map, temp) != false) {
            for (int i = 0; i < 100; i = i + 3) {
                player[i] = player[i] - 1.0f;
            }
        }
    }
}
```

Εικόνα 3.17: Κλήση των συναρτήσεων στην moveSquare

4 ΕΡΩΤΗΜΑ ΙΙΙ: ΘΗΣΑΥΡΟΣ

4.1 ΟΡΙΣΜΟΣ BUFFER ΓΙΑ ΤΟΝ ΘΗΣΑΥΡΟ

Σχετικά με τον θησαυρό, ορίσαμε τον πίνακα **treasure_buffer** με τον οποίο η θέση του αρχικοποιείται στην γραμμή 2 και στήλη 6 στον χάρτη του λαβυρίνθου. Εκεί θα είναι η πρώτη του εμφάνιση κάθε φορά που αρχίζει το πρόγραμμα. Δοκιμάσαμε τρόπους για να εμφανίζεται τυχαία και στην αρχή αλλά αυτό μας δημιούργησε θέμα με το κάλεσμα της **glDrawArrays()** και το αφήσαμε. Οι τιμές των συντεταγμένων x,y και z στον πίνακα έχουν οριστεί έτσι ώστε ο κύβος του θησαυρού να έχει μήκος 0,8 όπως ζητείται. Παρακάτω ακολουθεί ο ορισμός αυτού του πίνακα.

```
2886 static GLfloat treasure_buffer[] = {
2887
2888     //bottom
2889     0.1f, 3.9f, 0.10f, //xmin,ymax point
2890     0.1f, 3.1f, 0.10f,
2891     0.9f, 3.9f, 0.10f,
2892     0.1f, 3.1f, 0.10f,
2893     0.9f, 3.9f, 0.10f,
2894     0.9f, 3.1f, 0.10f,
2895     //up
2896     0.1f, 3.9f, 0.90f,
2897     0.1f, 3.1f, 0.90f,
2898     0.9f, 3.9f, 0.90f,
2899     0.1f, 3.1f, 0.90f,
2900     0.9f, 3.9f, 0.90f,
2901     0.9f, 3.1f, 0.90f,
2902     //front
2903     0.1f, 3.1f, 0.10f,
2904     0.1f, 3.1f, 0.90f,
2905     0.9f, 3.1f, 0.10f,
2906     0.9f, 3.1f, 0.10f,
2907     0.9f, 3.1f, 0.90f,
2908     0.1f, 3.1f, 0.90f,
2909
2910     //back
2911     0.1f, 3.9f, 0.10f,
2912     0.1f, 3.9f, 0.90f,
2913     0.9f, 3.9f, 0.10f,
2914     0.9f, 3.9f, 0.10f,
2915     0.1f, 3.9f, 0.90f,
2916     //right
2917     0.9f, 3.1f, 0.10f,
2918     0.9f, 3.1f, 0.90f,
2919     0.9f, 3.9f, 0.10f,
2920     0.9f, 3.9f, 0.10f,
2921     0.9f, 3.9f, 0.90f,
2922     0.9f, 3.1f, 0.90f,
2923     //left
2924     0.1f, 3.1f, 0.10f,
2925     0.1f, 3.1f, 0.90f,
2926     0.1f, 3.9f, 0.10f,
2927     0.1f, 3.9f, 0.10f,
2928     0.1f, 3.9f, 0.90f,
2929     0.1f, 3.1f, 0.90f
2930
2931 };
```

Εικόνα 4.1: Ο πίνακας `treasure_buffer`

Έπειτα, πρέπει φυσικά να ορίσουμε το VBO για τον θησαυρό και να το κάνουμε bind με τον πίνακα μέσα στο loop επειδή η θέση του θα αλλάζει. Μετά καλούμε την `glDrawArrays()` με όρισμα 36 επειδή έχουμε πάλι έναν κύβο.

```
3100 GLuint treasurebuffer;
3101 glGenBuffers(1, &treasurebuffer);
3102 glBindBuffer(GL_ARRAY_BUFFER, treasurebuffer);
```

Εικόνα 4.2: Ορισμός VBO για τον θησαυρό

```
3215 //treasure
3216 glUniform1i(glGetUniformLocation(programID, "useTexture"), GL_TRUE);
3217 glEnableVertexAttribArray(0);
3218 glBindBuffer(GL_ARRAY_BUFFER, treasurebuffer);
3219 glBufferData(GL_ARRAY_BUFFER, sizeof(treasure_buffer), treasure_buffer, GL_STATIC_DRAW);
3220 glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 0, (void*)0);
```

Εικόνα 4.3: Εντολές για σχεδιασμό εντός του main loop

Η πρώτη εντολή έχει να κάνει με το texture και θα την εξηγήσουμε αργότερα.

4.2 ΕΜΦΑΝΙΣΗ ΘΗΣΑΥΡΟΥ ΣΕ ΤΥΧΑΙΑ ΘΕΣΗ

Έχουμε επιλέξει όταν ο παίκτης ακουμπάει τον θησαυρό, αυτός να εξαφανίζεται και έπειτα να μην εμφανίζεται ποτέ ξανά. Η συνάρτηση **spawnTreasure()** κάνει την κύρια δουλειά για την εμφάνιση του θησαυρού σε τυχαία θέση. Παίρνει ως όρισμα την τρέχουσα θέση του θησαυρού, τον χάρτη του λαβυρίνθου και τη θέση του παίκτη. Το **treasureStartingPoint** είναι πάντα ο βοηθητικός πίνακας **treasure_const** που έχουμε ορίσει. Αυτός ο πίνακας ορίζει μια σταθερή θέση του κύβου στο (0,0) και σε αυτόν γίνονται οι πράξεις για αλλαγή της θέσης του θησαυρού.

```
2839 static GLfloat treasure_const[] = {
2840
2841     //bottom
2842     0.1f, 0.9f, 0.10f, //xmin,ymax point
2843     0.1f, 0.1f, 0.10f,
2844     0.9f, 0.9f, 0.10f,
2845     0.1f, 0.1f, 0.10f,
2846     0.9f, 0.9f, 0.10f,
2847     0.9f, 0.1f, 0.10f,
2848     //up
2849     0.1f, 0.9f, 0.90f,
2850     0.1f, 0.1f, 0.90f,
2851     0.9f, 0.9f, 0.90f,
2852     0.1f, 0.1f, 0.90f,
2853     0.9f, 0.9f, 0.90f,
2854     0.9f, 0.1f, 0.90f,
2855     //front
2856     0.1f, 0.1f, 0.10f,
2857     0.1f, 0.1f, 0.90f,
2858     0.9f, 0.1f, 0.10f,
2859     0.9f, 0.1f, 0.10f,
2860     0.9f, 0.1f, 0.90f,
2861     0.1f, 0.1f, 0.90f,
2862
2863     //back
2864     0.1f, 0.9f, 0.10f,
2865     0.1f, 0.9f, 0.90f,
2866     0.9f, 0.9f, 0.10f,
2867     0.9f, 0.9f, 0.10f,
2868     0.1f, 0.9f, 0.90f,
2869     //right
2870     0.9f, 0.1f, 0.10f,
2871     0.9f, 0.1f, 0.90f,
2872     0.9f, 0.9f, 0.10f,
2873     0.9f, 0.9f, 0.10f,
2874     0.9f, 0.9f, 0.90f,
2875     0.9f, 0.1f, 0.90f,
2876     //left
2877     0.1f, 0.1f, 0.10f,
2878     0.1f, 0.1f, 0.90f,
2879     0.1f, 0.9f, 0.10f,
2880     0.1f, 0.9f, 0.10f,
2881     0.1f, 0.9f, 0.90f,
2882     0.1f, 0.1f, 0.90f
2883 };
2884
```

Εικόνα 4.4: Ο πίνακας **treasure_const**

Στη συνάρτηση λοιπόν χρησιμοποιούμε την global μεταβλητή **spawning** για να ελέγξουμε αν επιτρέπεται να εμφανιστεί ο θησαυρός, αν ο παίκτης τον έχει ήδη συλλέξει θα είναι **false**. Στη συνέχεια με την εντολή **srand((unsigned)time(NULL));** εξασφαλίζουμε ότι το seed ορίζεται κάθε φορά με βάση το time και οι τυχαίες τιμές θα είναι διαφορετικές σε κάθε εκτέλεση του προγράμματος. Το **chooseRandomTexture** έχει να κάνει με την επιλογή τυχαίας εικόνας και θα το εξηγήσουμε πιο μετά. Στο while loop με την **rand()** αποθηκεύουμε μια τυχαία τιμή για το x και το y ανάμεσα από το -5 έως το 5 και προσθέτουμε αυτές τις τιμές στις θέσεις του βοηθητικού πίνακα. Αφού ο πίνακας αυτός αναφέρεται στη θέση (0,0) με αυτές τις πράξεις βρίσκουμε αμέσως μια νέα τυχαία θέση στο χάρτη. Αποθηκεύουμε αυτή τη θέση στον προσωρινό πίνακα **temp** και καλώντας την **checkTreasureMove()** ελέγχουμε αν είναι έγκυρη η κίνηση. Αν είναι, αλλάζουμε τον πίνακα του θησαυρού και θέτουμε το **valid** σε **true** για να σταματήσει το loop. Αλλιώς, το while loop συνεχίζει μέχρι να βρει μια έγκυρη θέση.


```
568 void spawnTreasure(GLfloat* treasureStartingPoint, GLfloat* treasure, GLfloat map[10][10], GLfloat* player) {
569
570     if (spawning == true) {
571
572         srand((unsigned)time(NULL));
573
574         // Toggle between textures
575         int chooseRandomTexture = 1 + (rand() % 4);
576         chooseTexture = chooseRandomTexture;
577
578
579         bool valid = false;
580         int randomX;
581         int randomY;
582
583         while (valid == false) {
584             randomX = -5 + (rand() % 10);
585
586             randomY = -5 + (rand() % 10);
587
588
589             GLfloat temp[108];
590             for (int i = 0; i < 108; i++) {
591                 temp[i] = treasure[i];
592             }
593
594             //x
595             for (int i = 0; i < 108; i += 3) {
596
597                 temp[i] = treasureStartingPoint[i] + (float)randomX;
598             }
599             //y
600             for (int i = 1; i < 108; i += 3) {
601
602                 temp[i] = treasureStartingPoint[i] + (float)randomY;
603             }
604
605
606             if (checkTreasureMove(map, temp, treasure, player)) {
607                 valid = true;
608
609                 for (int i = 0; i < 108; i++) {
610                     treasure[i] = temp[i];
611                 }
612             }
613         }
614     }
```

Εικόνα 4.5: Η συνάρτηση spawnTreasure

Η συνάρτηση **checkTreasureMove()** λειτουργεί ακριβώς όπως και η **checkMove()** για τον χαρακτήρα με την μόνη διαφορά ότι παίρνει ως όρισμα και την τωρινή θέση του θησαυρού (**treasureOld**) για να ελέγξει αν η καινούργια (**treasureNew**) είναι ίση με την τωρινή καθώς θέλουμε να εμφανίζεται κάθε φορά σε διαφορετική θέση σε σχέση με την προηγούμενη. Επίσης, καλεί τις **upscale** για τον θησαυρό και τον χαρακτήρα, για να ελέγξει αν ο θησαυρός εμφανίζεται πάνω του. Έχουμε πει πως μέσω των **upscale** αποθηκεύουμε μόνο το (x_{min}, y_{max}) σημείο. Αν η απόλυτη διαφορά αυτών των x και y είναι μικρότερη του 1 σημαίνει ότι χαρακτήρας και θησαυρός βρίσκονται στο ίδιο σημείο και επιστρέφεται false. Μετά, η μετατροπή των σημείων σε σημεία του χάρτη για το κάθε τεταρτημόριο είναι και αυτή ίδια με του χαρακτήρα.

```
475 bool checkTreasureMove(GLfloat map[10][10], GLfloat* treasureNew, GLfloat* treasureOld, GLfloat* player) {
476     //check bottom side only
477     ;
478     ;
479     //check if player is out of map
480     for (int i = 0; i < 18; i = i + 3) {
481         if (treasureNew[i] > 5 || treasureNew[i] < -5) {
482             return false;
483         }
484     }
485     ;
486     //check if it spawns on the previous position
487     if (treasureNew[0] == treasureOld[0] && treasureNew[1] == treasureOld[1]) {
488         return false;
489     }
490     ;
491     ;
492     GLfloat* tempPlayer = upscalePlayer(player);
493     GLfloat* tempTreasure = upscaleTreasure(treasureNew);
494     ;
495     ;
496     ;
497     //check if it spawns on the player
498     if (abs(tempPlayer[0] - tempTreasure[0]) < 1 && abs(tempPlayer[1] - tempTreasure[1]) < 1) {
499         free(tempPlayer);
500         free(tempTreasure);
501         return false;
502     }
503     ;
504     ;
505     ;
506     ;
507     int row = 0;
508     int col = 0;
509     ;
```

Εικόνα 4.6: Η συνάρτηση checkTreasureMove (1)

```
537 //4th quarter
538 else if (tempTreasure[0] > 0.0f && tempTreasure[1] < 0.0f) {
539     row = -tempTreasure[1] + 5;
540     col = tempTreasure[0] + 5;
541     if (map[row][col] == 1.0f) {
542         free(tempPlayer);
543         free(tempTreasure);
544         return false;
545     }
546 }
547 //3rd quarter
548 else if (tempTreasure[0] < 0.0f && tempTreasure[1] < 0.0f) {
549     row = -tempTreasure[1] + 5;
550     col = tempTreasure[0] + 5;
551     if (map[row][col] == 1.0f) {
552         free(tempPlayer);
553         free(tempTreasure);
554         return false;
555     }
556 }
557 free(tempPlayer);
558 free(tempTreasure);
559 return true;
560 }
561 ;
562 ;
563 ;
564 ;
565 ;
566 ;
567 ;
568 ;
569 ;
570 //transform player coords to map cords
571 ;
572 //2nd quarter
573 if (tempTreasure[0] < 0.0f && tempTreasure[1] > 0.0f) {
574     row = -(tempTreasure[1] - 5);
575     col = tempTreasure[0] + 5;
576     if (map[row][col] == 1.0f) {
577         free(tempPlayer);
578         free(tempTreasure);
579         return false;
580     }
581 }
582 //1st quarter
583 else if (tempTreasure[0] > 0.0f && tempTreasure[1] > 0.0f) {
584     row = -tempTreasure[1] + 5;
585     col = tempTreasure[0] + 5;
586     if (map[row][col] == 1.0f) {
587         free(tempPlayer);
588         free(tempTreasure);
589         return false;
590     }
591 }
592 ;
593 ;
594 ;
595 ;
596 ;
```

Εικόνα 4.7: Η συνάρτηση checkTreasureMove (2)

Η συνάρτηση **upscaleTreasure()** λειτουργεί επίσης όπως ακριβώς η αντίστοιχη για τον χαρακτήρα, με μόνη διαφορά οι πράξεις στα σημεία να γίνονται κατά 0.1 αφού ο θησαυρός έχει διαφορετικές διαστάσεις και διαφέρει κατά 0.1 στην πάνω και κάτω πλευρά από ένα ολόκληρο μπλοκ του λαβυρίνθου. Σε αυτό το σημείο, να αναφέρουμε ότι αλλάξαμε στις δύο συναρτήσεις upscale τον ορισμό του προσωρινού πίνακα **temp** σε σχέση με τις προηγούμενες ασκήσεις. Τον δημιουργούμε με malloc ώστε να δεσμεύσουμε χώρο μνήμης για τη χρήση του, καθώς όταν τον ορίζαμε τοπικά όπως πριν και καλούσαμε μαζί τις upscale μάλλον υπήρχε θέμα με τους δείκτες και οι πίνακες έδειχναν πάντα στο ίδιο σημείο με αποτέλεσμα να έχουν τα ίδια στοιχεία ακόμα και όταν ο θησαυρός δεν βρισκόταν στην ίδια θέση με τον παίκτη. Για αυτό το λόγο κάνουμε free στην **checkTreasureMove()** πριν επιστρέψει και τερματίζει η κλήση της σε κάθε περίπτωση.

```
388  GLfloat* upscaleTreasure(GLfloat a[]) {
389      //upscale bottom side
390
391      GLfloat* temp = (GLfloat*)malloc(2 * sizeof(GLfloat));
392      GLfloat x = a[0];
393      GLfloat y = a[1];
394      //x
395      if (a[0] < 0) {
396          temp[0] = x - 0.1f;
397      }
398      else {
399          temp[0] = x + 0.1f; //???
400      }
401      //y
402      if (a[1] < 0) {
403          temp[1] = y - 0.1f; //???
404      }
405      else {
406          temp[1] = y + 0.1f;
407      }
408      return temp;
409  }
```

Εικόνα 4.8: Η συνάρτηση upscaleTreasure

Τέλος, αυτό που μας μένει να κάνουμε είναι να εφαρμόσουμε ένα είδος καθυστέρησης στην εμφάνιση του θησαυρού για να εμφανίζεται κάπου τυχαία ανά τακτά χρονικά διαστήματα. Έτσι ορίζουμε πριν το main loop στην γραμμή 3129 τις μεταβλητές **treasureTimer** για τον «χρόνο» που έχει περάσει κάθε φορά, **deltaTime** για να αυξάνουμε κάθε φορά τον χρόνο κατά 1 και **spawnInterval** για τον μέγιστο χρόνο που θα περιμένουμε. Μετά εντός του loop που τρέχει συνεχώς στο πρόγραμμα, στην γραμμή 3276 αυξάνουμε κάθε φορά το **treasureTimer** και αν έχει γίνει ίσο με τον μέγιστο χρόνο καθυστέρησης, εμφανίζουμε τον θησαυρό και θέτουμε το timer ξανά στο 0. Αυτό το μπλοκ κώδικα τρέχει επανειλημμένα μέσα στο βρόγχο και εξασφαλίζει ότι ο θησαυρός επανεμφανίζεται ανά ίσα χρονικά διαστήματα σε άλλη τυχαία θέση.

```
3275
3276  treasureTimer += deltaTime;          // Add the time between frames
3277  if (treasureTimer >= spawnInterval) {
3278      spawnTreasure(treasure_const, treasure_buffer, map, character_buffer);
3279      treasureTimer = 0.0f;             // Reset the timer
3280  }
3281
```

Εικόνα 4.9: Μπλοκ κώδικα για την εμφάνιση του θησαυρού

4.3 ΕΦΑΡΜΟΓΗ ΥΦΗΣ ΣΤΟΝ ΘΗΣΑΥΡΟ

Για να μπορέσουμε να εφαρμόσουμε texture στον κύβο του θησαυρού πρέπει να επεκτείνουμε τα shader. Αρχικά, στον vertex shader προσθέτουμε την εντολή **layout(location = 2) in vec2 vertexUV;** ώστε να θέσουμε την λειτουργία του texture στο attribute 2. Επίσης προσθέτουμε **out vec2 UV;** για να γίνει input στον fragment shader και εντός της main γράφουμε **UV = vertexUV;** ώστε να ξέρει ο shader που να το αντιστοιχίσει.

```
1  #version 330 core
2
3  layout(location = 0) in vec3 vertexPosition_modelspace;
4  layout(location = 1) in vec4 vertexColor;
5  //uv
6  layout(location = 2) in vec2 vertexUV;
7  // light
8  layout(location = 3) in vec3 vertexNormal_modelspace;
9
10
11
12  out vec4 fragmentColor;
13  //uv
14  out vec2 UV;
15
16  out vec3 Position_worldspace;
17  out vec3 Normal_cameraspace;
18  out vec3 EyeDirection_cameraspace;
19  out vec3 LightDirection_cameraspace;
20
21  uniform mat4 MVP;
22  uniform mat4 V;
23  uniform mat4 M;
24  uniform vec3 LightPosition_worldspace;
25
26  void main()
27  {
28      gl_Position = MVP * vec4(vertexPosition_modelspace,1);
29      fragmentColor = vertexColor;
30      //uv
31      UV = vertexUV;
32      Position_worldspace = (M * vec4(vertexPosition_modelspace,1)).xyz;
```

Εικόνα 4.10: Ο vertex shader

Οι υπόλοιπες εντολές έχουν να κάνουν με την υλοποίηση του μοντέλου φωτισμού phong και θα αναλυθούν στην αντίστοιχη παράγραφο.

Στον fragment shader η δουλειά μας είναι πιο περίπλοκη. Περνάμε ως input το **in vec2 UV;** από το vertex shader και ως output το **out vec3 color;** και ορίζουμε τις uniform μεταβλητές **uniform sampler2D myTextureSampler;** για να μπορέσουμε να χρησιμοποιήσουμε το texture και **uniform bool useTexture;**. Το πρόβλημα εδώ είναι ότι ο fragment shader μπορεί να έχει μόνο μια έξοδο και εμείς τον χρειαζόμαστε για να θέτουμε χρώμα στα σχήματά μας αλλά και texture. Επομένως, πρέπει να ελέγξουμε με ένα if αν ως output θα χρησιμοποιήσουμε χρώμα ή texture. Η μεταβλητή **useTexture** θα αλλάζει μέσα από το πρόγραμμα. Αν είναι true θα χρησιμοποιηθεί texture για το αντικείμενο, ενώ αν είναι false θα χρησιμοποιηθεί χρώμα. Επειδή έχουμε υλοποιήσει και φωτισμό στο πρόγραμμά μας το **MaterialDiffuseColor** θα αλλάξει τιμή με βάση αυτό και

τελικά το **color** που βγαίνει ως output παίρνει την κατάλληλη τιμή του. Πιο πολλές λεπτομέρειες σχετικά με αυτό στην αντίστοιχη παράγραφο.

```
1  #version 330 core
2
3  in vec4 fragmentColor;
4  in vec2 UV;
5
6  in vec3 Position_worldspace;
7  in vec3 Normal_cameraspace;
8  in vec3 EyeDirection_cameraspace;
9  in vec3 LightDirection_cameraspace;
10
11 out vec3 color;
12
13 uniform sampler2D myTextureSampler;
14 uniform bool useTexture; // True for textured objects, false for color-only objects
15 uniform mat4 MV;
16 uniform vec3 LightPosition_worldspace;
17
18 void main() {
19     // Light properties
20     vec3 LightColor = vec3(1.0, 1.0, 1.0);
21     float LightPower = 50.0f;
22
23     // Material properties
24     vec3 MaterialDiffuseColor;
25     if (useTexture) {
26         // Use texture color for textured objects
27         MaterialDiffuseColor = texture(myTextureSampler, UV).rgb;
28     } else {
29         // Use flat fragment color for non-textured objects
30         MaterialDiffuseColor = fragmentColor.rgb;
31     }
32 }
```

Εικόνα 4.11: Ο fragment shader

Για να μπορέσουμε να φορτώσουμε την υφή, πρέπει φυσικά να ορίσουμε και τις UV συντεταγμένες τις εικόνες. Αυτό κάνουμε με τον πίνακα **g_uvbuffer_data** όπου αντιστοιχίζουμε το κάθε τρίγωνο της κάθε πλευράς με τα κατάλληλα σημεία της εικόνας. Για την εικόνα ξέρουμε ότι το (x_{min}, y_{min}) είναι το σημείο $(0,0)$ και το (x_{max}, y_{max}) είναι το σημείο $(1,1)$ και θέλουμε ολόκληρη η εικόνα να εμφανίζεται σε κάθε πλευρά. Ας δούμε πως ορίσαμε τις συντεταγμένες για την κάτω πλευρά του κύβου: Στον πίνακα **treasure_buffer** για το πρώτο τρίγωνο έχουμε με τη σειρά ότι το πρώτο σημείο είναι το πάνω αριστερά, άρα το αντιστοιχούμε με το $(0,1)$ της εικόνας, το επόμενο είναι το κάτω αριστερά και πάει με το $(0,0)$ της εικόνας και το άλλο είναι το πάνω δεξιά και πάει με το $(1,1)$. Για το δεύτερο τρίγωνο έχουμε το πρώτο σημείο ως κάτω αριστερά με το $(0,0)$, το επόμενο είναι το πάνω δεξιά και αντιστοιχίζεται με το $(1,1)$ και το άλλο το κάτω δεξιά και πάει με το $(1,0)$. Με παρόμοιο τρόπο ορίζουμε τις συντεταγμένες για κάθε άλλη πλευρά.

```
2933 static GLfloat g_uvbuffer_data[] = {
2934     //αντιστοιχίζουμε το κάθε τρίγωνο της κάθε πλευρας
2935     //με τα σωστά σημεία της εικόνας
2936
2937     //first block - bottom
2938     0.0f, 1.0f,
2939     0.0f, 0.0f,
2940     1.0f, 1.0f,
2941     0.0f, 0.0f,
2942     1.0f, 1.0f,
2943     1.0f, 0.0f,
2944     //second block - up
2945     0.0f, 1.0f,
2946     0.0f, 0.0f,
2947     1.0f, 1.0f,
2948     0.0f, 0.0f,
2949     1.0f, 1.0f,
2950     1.0f, 0.0f,
2951     //third block - front
2952     0.0f, 0.0f,
2953     0.0f, 1.0f,
2954     1.0f, 0.0f,
2955     1.0f, 0.0f,
2956     1.0f, 1.0f,
2957     0.0f, 1.0f,
2958
2959     //fourth block - back
2960     0.0f, 0.0f,
2961     0.0f, 1.0f,
2962     1.0f, 0.0f,
2963     1.0f, 1.0f,
2964     0.0f, 1.0f,
2965     //fifth block - right
2966     0.0f, 0.0f,
2967     0.0f, 1.0f,
2968     1.0f, 0.0f,
2969     1.0f, 0.0f,
2970     1.0f, 1.0f,
2971     0.0f, 1.0f,
2972     //sixth block - left
2973     0.0f, 0.0f,
2974     0.0f, 1.0f,
2975     1.0f, 0.0f,
2976     1.0f, 0.0f,
2977     1.0f, 1.0f,
2978     0.0f, 1.0f,
2979 };
```

Εικόνα 4.12: Ο πίνακας `g_uvbuffer_data`

Αφού τελειώσαμε με τον ορισμό των UV, ορίζουμε το VBO `uvbuffer` για να κάνουμε bind τον πίνακα (εικόνα 4.13) και μέσα στο loop ενεργοποιούμε το attribute 2 στον θησαυρό για να πάρει την υφή που θέλουμε (εικόνα 4.14). Εκεί επιλέγεται και ποια εικόνα θα πάρει τυχαία κάθε φορά αλλά αυτό θα το εξηγήσουμε αναλυτικά παρακάτω.

```
3094
3095 GLuint uvbuffer;
3096 glGenBuffers(1, &uvbuffer);
3097 glBindBuffer(GL_ARRAY_BUFFER, uvbuffer);
3098 glBufferData(GL_ARRAY_BUFFER, sizeof(g_uvbuffer_data), g_uvbuffer_data, GL_STATIC_DRAW);
```

Εικόνα 4.13: Ορισμός VBO των UV συντεταγμένων

```
3227
3228 // texture
3229 glEnableVertexAttribArray(2);
3230 glBindBuffer(GL_ARRAY_BUFFER, uvbuffer);
3231 glVertexAttribPointer(2, 2, GL_FLOAT, GL_FALSE, 0, (void*)0);
3232 if (chooseTexture == 1) {
3233     // Use the first texture
3234     glActiveTexture(GL_TEXTURE0);
3235     glBindTexture(GL_TEXTURE_2D, textureID);
3236     glUniform1i(glGetUniformLocation(programID, "myTextureSampler"), 0);
3237 }
```

Εικόνα 4.14: Binding των UVs στον θησαυρό

Τέλος, φορτώνουμε την εικόνα στο πρόγραμμα μέσω της `stbi_load`.

```
3028     int width;
3029     int height;
3030     int numColCh;
3031     unsigned char* dataImage = stbi_load("coins.jpg", &width, &height, &numColCh, 0);
3032     // Create one OpenGL texture
3033     GLuint textureID;
3034     glGenTextures(1, &textureID);
3035
3036     //Bind the newly created texture
3037     //glActiveTexture(GL_TEXTURE0);
3038     glBindTexture(GL_TEXTURE_2D, textureID);
3039
3040     //Give the image to OpenGL
3041     glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, width, height, 0, GL_RGB, GL_UNSIGNED_BYTE, dataImage);
3042     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
3043     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
3044     stbi_image_free(dataImage);
```

Εικόνα 4.15: Φόρτωση εικόνας

Πριν τελειώσουμε όμως, όπως είχαμε αναφέρει πριν στον fragment shader, πριν πάμε να σχεδιάσουμε κάποιο αντικείμενο πρέπει να θέσουμε την τιμή της uniform μεταβλητής του shader **useTexture** ως true ή false ανάλογα αν θέλουμε να δώσουμε texture ή χρώμα. Για αυτό καλούμε **glUniform1i(glGetUniformLocation(programID, "useTexture"), GL_FALSE);** για τον λαβύρινθο και για τον χαρακτήρα και **glUniform1i(glGetUniformLocation(programID, "useTexture"), GL_TRUE);** για τον θησαυρό.

5 ΕΡΩΤΗΜΑ IV: ΑΠΟΚΤΗΣΗ ΘΗΣΑΥΡΟΥ

Όταν ο χρήστης ακουμπήσει τον θησαυρό, αυτός πρέπει να μικρύνει για λίγο στη μέση και μετά να εξαφανιστεί. Στη γραμμή 3334 ελέγχουμε αν η συνάρτηση **checkIfPlayerTouched()** είναι true και η μεταβλητή **treasureCollected** είναι false. Αν ναι, η global μεταβλητή **spawning** θα γίνει false ώστε να πάψει να εμφανίζεται ο θησαυρός, εκτελείται ο ήχος και αλλάζει το background (λεπτομέρειες στην αντίστοιχη παράγραφο) και συρρικνώνεται ο θησαυρός καλώντας την **reduceSizeOfTreasure()**. Αν εισέλθουμε λοιπόν μέσα στο if η **treasureCollected** θα γίνει true και αυτό το κομμάτι κώδικα δεν θα εκτελεστεί ξανά. Έτσι καταφέραμε να λύσουμε το πρόβλημα του θησαυρού να συρρικνώνεται συνεχώς στο πρόγραμμα. Οι υλοποιήσεις των συναρτήσεων **checkIfPlayerTouched()** και **reduceSizeOfTreasure()** φαίνονται στις εικόνες 5.3 και 5.4 αντίστοιχα.

```
3333
3334     if (checkIfPlayerTouched(treasure_buffer, character_buffer) == true && treasureCollected == false) {
3335         spawning = false;
3336         treasureCollected = true;
3337         PlaySound(TEXT("coin.wav"), NULL, SND_FILENAME | SND_ASYNC);
3338         glClearColor(1.0f, 0.5f, 0.0f, 0.0f);
3339         reduceSizeOfTreasure(treasure_buffer);
3340     }
3341 }
```

Εικόνα 5.1: Μπλοκ κώδικα για τον έλεγχο της επαφής

Στη γραμμή 3138 ορίσαμε κάποιες μεταβλητές για να δώσουμε κάποιο delay στην εξαφάνιση του θησαυρού αφού έχει ήδη συρρικνωθεί. Η **despawnTimer** μετράει τον «χρόνο» μέχρι να εξαφανιστεί και η **treasureDespawnInterval** θέτει τον μέγιστο αυτό χρόνο. Από την γραμμή 3265 λοιπόν, αρχίζει να αυξάνεται η **despawnTimer** κάθε φορά που τρέχει το main loop και όταν γίνει ίση με τον μέγιστο χρόνο, όλα τα σημεία του θησαυρού θέτονται στο 0 ώστε να εξαφανιστεί. Αυτό το συγκεκριμένο μπλοκ εντολών εξασφαλίζει ότι ο συρρικνωμένος θησαυρός θα παραμείνει για ένα διάστημα ορατός.

```
3264
3265     if (treasureCollected) {
3266         despawnTimer += deltaTime;
3267         if (despawnTimer >= treasureDespawnInterval) {
3268             glClearColor(0.0f, 0.0f, 0.0f, 0.0f);
3269             for (int i = 0; i < 108; i++) {
3270                 treasure_buffer[i] = 0.0f;
3271             }
3272         }
3273     }
```

Εικόνα 5.2: Μπλοκ κώδικα για την εξαφάνιση του θησαυρού

Τώρα ας δούμε αναλυτικά την **checkIfPlayerTouched()**. Παρόμοια με την **checkTreasureMove()**, καλούμε τις **upscale** για να ελέγξουμε αν ο θησαυρός βρίσκεται στην ίδια θέση με τον παίκτη αυτή τη φορά. Δηλαδή, αν η απόλυτη διαφορά των (x_{min} , y_{max}) σημείων τους είναι μικρότερη από 0.5 σημαίνει ότι βρίσκονται στο ίδιο σημείο, η **playersTouch** γίνεται true και μετά επιστρέφεται.

```
617
618 bool checkIfPlayerTouched(GLfloat* treasure, GLfloat* player) {
619     GLfloat* tempPlayer = upscalePlayer(player);
620     GLfloat* tempTreasure = upscaleTreasure(treasure);
621
622     bool playersTouch = false;
623     if (abs(player[0] - treasure[0]) <= 0.5 && abs(player[1] - treasure[1]) <= 0.5) { //meaning that it touched the treasure
624         playersTouch = true;
625     }
626     free(tempPlayer);
627     free(tempTreasure);
628     return playersTouch;
629 }
```

Εικόνα 5.3: Η συνάρτηση **checkIfPlayerTouched**

Η συνάρτηση **reduceSizeOfTreasure()** μειώνει το μέγεθος του κύβου στο μισό. Αρχικά, θέτουμε τις ελάχιστες και μέγιστες συντεταγμένες των x,y,z σε κάποια τυχαία συντεταγμένη. Επιλέξαμε το πρώτο σημείο του πίνακα του θησαυρού. Μετά στον βρόγχο for βρίσκουμε τα μέγιστα και ελάχιστα σημεία διατρέχοντας όλο τον πίνακα για κάθε συντεταγμένη και αν βρούμε μια μεγαλύτερη (μικρότερη) από το τρέχων max (min), θέτουμε αυτή ως τη μέγιστη (ελάχιστη). Στο τέλος θα έχουμε βρει τα μέγιστα και ελάχιστα σημεία, δηλαδή τα όρια του κύβου. Έπειτα υπολογίζουμε το κέντρο του κύβου και με βάση αυτό μειώνουμε κάθε σημείο στο μισό. Πιο συγκεκριμένα, ουσιαστικά προσθέτουμε το μισό του κέντρου και το μισό του αρχικού σημείου και συνολικά τα σημεία μας κάνουν τον μισό κύβο, αφού αν απλά διαιρέσουμε όλα τα σημεία στη μέση θα πάρουμε το $\frac{1}{4}$ του κύβου.


```
631 void reduceSizeOfTreasure(GLfloat* treasure_buffer) {  
632     // Find the bounds of the treasure  
633     GLfloat xmin = treasure_buffer[0], xmax = treasure_buffer[0];  
634     GLfloat ymin = treasure_buffer[1], ymax = treasure_buffer[1];  
635     GLfloat zmin = treasure_buffer[2], zmax = treasure_buffer[2];  
636  
637     for (int i = 0; i < 36 * 3; i += 3) {  
638         if (treasure_buffer[i] < xmin) xmin = treasure_buffer[i];  
639         if (treasure_buffer[i] > xmax) xmax = treasure_buffer[i];  
640         if (treasure_buffer[i + 1] < ymin) ymin = treasure_buffer[i + 1];  
641         if (treasure_buffer[i + 1] > ymax) ymax = treasure_buffer[i + 1];  
642         if (treasure_buffer[i + 2] < zmin) zmin = treasure_buffer[i + 2];  
643         if (treasure_buffer[i + 2] > zmax) zmax = treasure_buffer[i + 2];  
644     }  
645  
646     // Calculate the center  
647     GLfloat xcenter = (xmin + xmax) / 2.0f;  
648     GLfloat ycenter = (ymin + ymax) / 2.0f;  
649     GLfloat zcenter = (zmin + zmax) / 2.0f;  
650  
651     // Scale each vertex toward the center  
652     for (int i = 0; i < 36 * 3; i += 3) {  
653         treasure_buffer[i] = xcenter + 0.5f * (treasure_buffer[i] - xcenter);  
654         treasure_buffer[i + 1] = ycenter + 0.5f * (treasure_buffer[i + 1] - ycenter);  
655         treasure_buffer[i + 2] = zcenter + 0.5f * (treasure_buffer[i + 2] - zcenter);  
656     }  
657 }
```

Εικόνα 5.4: Η συνάρτηση reduceSizeOfTreasure

6 ΕΡΩΤΗΜΑ V: ΡΥΘΜΙΣΕΙΣ ΚΑΜΕΡΑΣ

Στο συγκεκριμένο ερώτημα, η υλοποίηση των πρώτων τριών bullet points είναι ίδια με την προηγούμενη άσκηση. Αυτό που καλούμαστε, αρχικά, να κάνουμε είναι να ρυθμίσουμε κατάλληλα την κάμερά μας. Έτσι, θα χρησιμοποιήσουμε την εντολή **lookAt()**. Η πρώτη παράμετρος αφορά την αρχική θέση της κάμερας, οπότε εκεί θα βάλουμε ένα διάνυσμα με τιμές (0.0, 0.0, 20.0).

```
// Camera matrix  
ViewMatrix = glm::lookAt(  
    glm::vec3(0.0f, 0.0f, 20.0f),  
    glm::vec3(0.0f, 0.0f, 0.25f),  
    glm::vec3(0.0f, 1.0f, 0.0f)); // Head is up
```

Εικόνα 6.1: Εντολή lookAt και ρύθμιση των παραμέτρων της

Η δεύτερη παράμετρος αφορά το προς τα πού κοιτάει, οπότε θα θέσουμε το διάνυσμα (0.0, 0.0, 0.25) ενώ η τρίτη παράμετρος αφορά το ανιόν διάνυσμα, το οποίο το θέτουμε ίσο με (0.0, 1.0, 0.0).

Τον πίνακα που επιστρέφει θα τον θέτουμε ίσο με το **ViewMatrix**.

Θέλουμε να μετακινούμε την κάμερα, ανάλογα με το πάτημα των κουμπιών. Ζητείται να μετακινείται γύρω από τους άξονες x και y αλλά να υλοποιήσουμε και μία λειτουργία zoom in/ zoom out. Τέλος ζητείται να γίνεται ranning στον άξονα x και y.

Έτσι, θα υλοποιήσουμε την συνάρτηση camera_function(). Για την κάθε λειτουργία θα χρησιμοποιήσουμε ένα if statement. Αρχίζοντας με το rotation στον άξονα x, ελέγχουμε για το πάτημα του κουμπιού W και αν διαπιστώσουμε πως πατήθηκε, τότε χρησιμοποιούμε την **rotate()** με παραμέτρους τον πίνακα **ViewMatrix**, την τιμή 0.025 η οποία είναι το πόσα radians θα μετακινηθούμε και τέλος ένα διάνυσμα (1, 0, 0) το οποίο εκφράζει τον άξονα περιστροφής. Τον πίνακα που επιστρέφει τον θέτουμε ίσο με το **ViewMatrix**. Ομοίως, κάνουμε παρακάτω έλεγχο για πάτημα του κουμπιού X και καλούμε την rotate με ίδιες παραμέτρους εκτός από τα radians όπου τώρα είναι -0.025.

```
void camera_function()
{
    if (glfwGetKey(window, GLFW_KEY_W) == GLFW_PRESS) { //up
        ViewMatrix = glm::rotate(ViewMatrix, 0.025f, glm::vec3(1, 0, 0));
    }
    else if (glfwGetKey(window, GLFW_KEY_X) == GLFW_PRESS) { //down
        ViewMatrix = glm::rotate(ViewMatrix, -0.025f, glm::vec3(1, 0, 0));
    }
    else if (glfwGetKey(window, GLFW_KEY_Q) == GLFW_PRESS) { //right
        ViewMatrix = glm::rotate(ViewMatrix, 0.025f, glm::vec3(0, 1, 0));
    }
    else if (glfwGetKey(window, GLFW_KEY_Z) == GLFW_PRESS) { //left
        ViewMatrix = glm::rotate(ViewMatrix, -0.025f, glm::vec3(0, 1, 0));
    }
    else if (glfwGetKey(window, GLFW_KEY_KP_ADD) == GLFW_PRESS) { // zoom in
        ZoomMatrix = glm::translate(glm::mat4(1.0f), glm::vec3(0, 0, 0.025f));
        tempMatrix = ZoomMatrix * ViewMatrix;
        if ((0 - tempMatrix[3][2]) >= 0.1) {
            ViewMatrix = ZoomMatrix * ViewMatrix;
        }
    }
    else if (glfwGetKey(window, GLFW_KEY_KP_SUBTRACT) == GLFW_PRESS) { // zoom out
        ZoomMatrix = glm::translate(glm::mat4(1.0f), glm::vec3(0, 0, -0.025f));
        tempMatrix = ZoomMatrix * ViewMatrix;
        if ((0 - tempMatrix[3][2]) <= 20.0) {
            ViewMatrix = ZoomMatrix * ViewMatrix;
        }
    }
}
```

Εικόνα 6.2: Συνάρτηση camera_function

Στο τρίτο else if, κάνουμε έλεγχο για το πάτημα του κουμπιού Q και έπειτα καλούμε την rotate με ίδιες παραμέτρους αλλά τώρα το διάνυσμα που εκφράζει τον άξονα περιστροφής είναι (0,1,0) εφόσον θέλουμε περιστροφή στον άξονα y. Παρακάτω ελέγχουμε για το πάτημα του κουμπιού Z και καλούμε την rotate ξανά με τιμή για την μετακίνηση -0.025. Στα δύο τελευταία else if, κάνουμε ελέγχους για πάτημα των κουμπιών + και - του NumPad και έπειτα καλούμε την translate (αντί της rotate) με ορίσματα τον μοναδιαίο πίνακα και ένα διάνυσμα που εκφράζει την μετακίνηση την οποία θέλουμε να πραγματοποιήσουμε (για αυτό βλέπουμε τιμές 0 στους άξονες x και y και 0.025, -0.025 στον άξονα z, διότι ο τρόπος που υλοποιήσαμε την λειτουργία zoom in/ zoom out είναι πως θα πραγματοποιείται μετακίνηση στον άξονα z).

Τον πίνακα που επιστρέφει η συνάρτηση translate τον θέτουμε ίσο με έναν πίνακα που ονομάζεται **ZoomMatrix** και είναι ουσιαστικά ένας προσωρινός βοηθητικός πίνακας που έχουμε ορίσει. Έπειτα, έχουμε έναν δεύτερο προσωρινό πίνακα που τον ονομάζουμε **tempMatrix** στον οποίο αποθηκεύουμε την τιμή που θα πάρει ο **ViewMatrix**. Κάνουμε έλεγχο ώστε να δούμε εάν η απόσταση από το αντικείμενό μας φτάσει να είναι πολύ μικρή (ή στην περίπτωση του zoom out, να είναι μεγάλη, δηλαδή η κάμερα να βρίσκεται στο αρχικό σημείο) τότε δεν κάνουμε ανάθεση στον **ViewMatrix** την τιμή. Τέλος, αν η απόσταση βρίσκεται μεταξύ αποδεκτών ορίων, και στις δύο περιπτώσεις θέτουμε το **ViewMatrix** ίσο με το γινόμενο του **ZoomMatrix** και της προηγούμενης τιμής του **ViewMatrix**.

```
glm::mat4 ViewMatrix;  
glm::mat4 ProjectionMatrix;  
glm::mat4 ZoomMatrix;
```

Εικόνα 6.3: Ορισμός ZoomMatrix

Τέλος για το panning, που είναι η επιπλέον λειτουργία μας, κάνουμε σχεδόν το ίδιο με το zoom, απλά τώρα χρησιμοποιούμε ένα διάνυσμα στην translate το οποίο έχει μηδέν τις τιμές y,z αν κινείται στον άξονα x και μηδέν τις τιμές x,z αν κινείται στον άξονα y.

```
else if (glfwGetKey(window, GLFW_KEY_G) == GLFW_PRESS) { //pan x left  
    panningMatrix = glm::translate(glm::mat4(1.0f), glm::vec3(0.025f, 0, 0));  
    ViewMatrix = panningMatrix * ViewMatrix;  
}  
else if (glfwGetKey(window, GLFW_KEY_H) == GLFW_PRESS) { //pan x right  
    panningMatrix = glm::translate(glm::mat4(1.0f), glm::vec3(-0.025f, 0, 0));  
    ViewMatrix = panningMatrix * ViewMatrix;  
}  
else if (glfwGetKey(window, GLFW_KEY_T) == GLFW_PRESS) { //pan y right  
    panningMatrix = glm::translate(glm::mat4(1.0f), glm::vec3(0, -0.025f, 0));  
    ViewMatrix = panningMatrix * ViewMatrix;  
}  
else if (glfwGetKey(window, GLFW_KEY_B) == GLFW_PRESS) { //pan y left  
    panningMatrix = glm::translate(glm::mat4(1.0f), glm::vec3(0, 0.025f, 0));  
    ViewMatrix = panningMatrix * ViewMatrix;  
}
```

Εικόνα 6.4: Panning της κάμερας

Ο λόγος που χρησιμοποιούμε τις τιμές 0.025 και -0.025 για τους μετασχηματισμούς είναι επειδή έπειτα από διάφορες δοκιμές που κάναμε καταλήξαμε πως αυτή είναι μία αρκετά καλή τιμή η οποία προσφέρει μία πιο “smooth” μετακίνηση.

Ολοκληρώνοντας, καλούμε την **camera_function()** μέσα στον βρόγχο do-while, πριν κάνουμε τον υπολογισμό για το MVP.

```
// Clear the screen  
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);  
  
// Use our shader  
glUseProgram(programID);  
  
camera_function();  
  
ProjectionMatrix = getProjectionMatrix();  
ViewMatrix = getViewMatrix();  
ZoomMatrix = ViewMatrix;  
glm::mat4 Model = glm::mat4(1.0f);  
  
glm::mat4 MVP = ProjectionMatrix * ViewMatrix * Model;
```

Εικόνα 6.5: Κλήση της camera_function()

7 ΕΡΩΤΗΣΗ BONUS : Α) ΔΙΑΦΟΡΕΤΙΚΕΣ ΥΦΕΣ

Στο συγκεκριμένο ερώτημα αυτό που έπρεπε να κάνουμε είναι να εφαρμόζεται ένα τυχαίο texture από τα 3 που έχουμε. Ο τρόπος που εργαστήκαμε είναι παρόμοιος με αυτόν που αναφέρθηκε και νωρίτερα, αλλά τώρα θα κάνουμε την ίδια διαδικασία για να εφαρμόσουμε και τα υπόλοιπα 2 textures που απαιτούνται πέρα από το πρώτο.

```
int width;
int height;
int numColCh;
unsigned char* dataImage = stbi_load("coins.jpg", &width, &height, &numColCh, 0);
// Create one OpenGL texture
GLuint textureID;
glGenTextures(1, &textureID);

//Bind the newly created texture
//glActiveTexture(GL_TEXTURE0);
glBindTexture(GL_TEXTURE_2D, textureID);

//Give the image to OpenGL
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, width, height, 0, GL_RGB, GL_UNSIGNED_BYTE, dataImage);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
stbi_image_free(dataImage);
// second photo
int width2;
int height2;
int numColCh2;
unsigned char* dataImage2 = stbi_load("coins2.jpeg", &width2, &height2, &numColCh2, 0);
// Create one OpenGL texture
GLuint textureID2;
glGenTextures(1, &textureID2);

//Bind the newly created texture
//glActiveTexture(GL_TEXTURE0);
glBindTexture(GL_TEXTURE_2D, textureID2);

//Give the image to OpenGL
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, width2, height2, 0, GL_RGB, GL_UNSIGNED_BYTE, dataImage2);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
stbi_image_free(dataImage2);
// third photo
int width3;
int height3;
int numColCh3;
unsigned char* dataImage3 = stbi_load("coins3.jpg", &width3, &height3, &numColCh3, 0);

// Create one OpenGL texture
GLuint textureID3;
glGenTextures(1, &textureID3);

//Bind the newly created texture
//glActiveTexture(GL_TEXTURE0);
glBindTexture(GL_TEXTURE_2D, textureID3);

//Give the image to OpenGL
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, width3, height3, 0, GL_RGB, GL_UNSIGNED_BYTE, dataImage3);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
stbi_image_free(dataImage3);
```

Εικόνα 7.1: Εισαγωγή των φωτογραφιών

Έχουμε ένα global variable **chooseTexture** το οποίο αρχικοποιείται στο μηδέν και έπειτα μέσα στην συνάρτηση *spawnTreasure()* λαμβάνει μία τυχαία τιμή με την χρήση της πράξης mod και με την χρήση της συνάρτησης **srand**.

```
// texture
glEnableVertexAttribArray(2);
glBindBuffer(GL_ARRAY_BUFFER, uvbuffer);
glVertexAttribPointer(2, 2, GL_FLOAT, GL_FALSE, 0, (void*)0);
if (chooseTexture == 1) {
    // Use the first texture
    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_2D, textureID);
    glUniform1i(glGetUniformLocation(programID, "myTextureSampler"), 0);
}
else if (chooseTexture == 2) {
    // Use the second texture
    glActiveTexture(GL_TEXTURE1);
    glBindTexture(GL_TEXTURE_2D, textureID2);
    glUniform1i(glGetUniformLocation(programID, "myTextureSampler"), 1);
}
else if (chooseTexture == 3) {
    // Use the second texture
    glActiveTexture(GL_TEXTURE2);
    glBindTexture(GL_TEXTURE_2D, textureID3);
    glUniform1i(glGetUniformLocation(programID, "myTextureSampler"), 2);
}
```

Εικόνα 7.2: Επιλογή του κατάλληλου texture

```
srand((unsigned)time(NULL));

// Toggle between textures
int chooseRandomTexture = 1 + (rand() % 4);
chooseTexture = chooseRandomTexture;
```

Εικόνα 7.3: Δημιουργία τυχαίας τιμής chooseTexture

Έτσι, κάθε φορά που πάει να μετακινηθεί ο θησαυρός επιλέγεται τυχαία ένα texture.

8 ΕΡΩΤΗΜΑ BONUS: Β) ΟΠΤΙΚΑ ΚΑΙ ΗΧΗΤΙΚΑ ΕΦΕ

Στο συγκεκριμένο ερώτημα καλούμαστε να υλοποιήσουμε ένα οπτικό εφέ και ένα ηχητικό εφέ για την στιγμή που ακουμπήσει τον θησαυρό ο χαρακτήρας μας. Για το οπτικό εφέ αποφασίσαμε να αλλάζουμε το χρώμα του background στιγμιαία και έτσι να δημιουργείται ένα εφέ «αστραπής», ενώ για τον ηχητικό έχουμε έναν ήχο «8-bit coin».

```
if (checkIfPlayerTouched(treasure_buffer, character_buffer, treasure_const) == true && treasureCollected == false) {  
    spawning = false;  
    treasureCollected = true;  
    PlaySound(TEXT("coin.wav"), NULL, SND_FILENAME | SND_ASYNC);  
    glClearColor(1.0f, 0.5f, 0.0f, 0.0f);  
    reduceSizeOfTreasure(treasure_buffer);  
}
```

Εικόνα 8.1: Ενεργοποίηση εφέ

Έτσι λοιπόν, στον παραπάνω κώδικα, βλέπουμε πως ελέγχεται η συνθήκη για το αν ακουμπάει ο χαρακτήρας τον κύβο και αν ισχύει αυτό τότε ξεκινάει να παίζει ο ήχος με την συνάρτηση **PlaySound()** και αλλάζει το χρώμα του background.

Σε άλλο σημείο του κώδικα υπάρχει και το παρακάτω κομμάτι ώστε το χρώμα να επιστρέφει στο αρχικό.

```
if (treasureCollected) {  
    despawnTimer += deltaTime;  
    if (despawnTimer >= treasureDespawnInterval) {  
        glClearColor(0.0f, 0.0f, 0.0f, 0.0f);  
        for (int i = 0; i < sizeof(treasure_buffer) / sizeof(treasure_buffer[0]); ++i) {  
            treasure_buffer[i] = 0.0f;  
        }  
    }  
}
```

Εικόνα 8.2: Επιστροφή χρώματος σε μαύρο

9 ΕΡΩΤΗΜΑ BONUS: Γ) ΜΟΝΤΕΛΟ ΦΩΤΙΣΜΟΥ PHONG

Η υλοποίηση του φωτισμού της σκηνής μας με το μοντέλο phong, πέρα από τις κατάλληλες αλλαγές στα shader απαιτεί και τον ορισμό των normals για την κάθε πλευρά των αντικειμένων μας. Ορίζουμε τον πίνακα **normals** (εικόνα 9.1) ο οποίος αποθηκεύει τα normals των πλευρών ενός κύβου. Αυτόν χρησιμοποιούμε για τον χαρακτήρα και τον θησαυρό.

```
2981 static GLfloat normals[] = {
2982
2983     //bottom
2984     0.0f, 0.0f, -1.0f,
2985     0.0f, 0.0f, -1.0f,
2986     0.0f, 0.0f, -1.0f,
2987     0.0f, 0.0f, -1.0f,
2988     0.0f, 0.0f, -1.0f,
2989     0.0f, 0.0f, -1.0f,
2990     //up
2991     0.0f, 0.0f, 1.0f,
2992     0.0f, 0.0f, 1.0f,
2993     0.0f, 0.0f, 1.0f,
2994     0.0f, 0.0f, 1.0f,
2995     0.0f, 0.0f, 1.0f,
2996     0.0f, 0.0f, 1.0f,
2997     //front
2998     0.0f, 1.0f, 0.0f,
2999     0.0f, 1.0f, 0.0f,
3000     0.0f, 1.0f, 0.0f,
3001     0.0f, 1.0f, 0.0f,
3002     0.0f, 1.0f, 0.0f,
3003     0.0f, 1.0f, 0.0f,
3004
3005     //back
3006     0.0f, -1.0f, 0.0f,
3007     0.0f, -1.0f, 0.0f,
3008     0.0f, -1.0f, 0.0f,
3009     0.0f, -1.0f, 0.0f,
3010     0.0f, -1.0f, 0.0f,
3011     0.0f, -1.0f, 0.0f,
3012     //right
3013     1.0f, 0.0f, 0.0f,
3014     1.0f, 0.0f, 0.0f,
3015     1.0f, 0.0f, 0.0f,
3016     1.0f, 0.0f, 0.0f,
3017     1.0f, 0.0f, 0.0f,
3018     1.0f, 0.0f, 0.0f,
3019     //left
3020     -1.0f, 0.0f, 0.0f,
3021     -1.0f, 0.0f, 0.0f,
3022     -1.0f, 0.0f, 0.0f,
3023     -1.0f, 0.0f, 0.0f,
3024     -1.0f, 0.0f, 0.0f,
3025     -1.0f, 0.0f, 0.0f,
3026 };
3027
```

Εικόνα 9.1: Ο πίνακας normals

Για κάθε πλευρά λοιπόν πρέπει να βρούμε το κάθετο διάνυσμα που προσπίπτει πάνω της. Τα σημεία επαναλαμβάνονται 6 φορές σε κάθε πλευρά γιατί έχουμε δυο τρίγωνα και 6 σημεία, που το καθένα θα πρέπει να αντιστοιχηθεί με το διάνυσμα για να γνωρίζει ο shader σε ποια πλευρά αναφέρεται. Έτσι, η κάτω πλευρά έχει normal το (0,0,-1) αφού βρίσκεται προς το -z, η πάνω το (0,0,1) αφού βρίσκεται προς το +z, η μπροστινή πλευρά το (0,1,0) αφού βρίσκεται προς το +y, η πίσω πλευρά το (0,-1,0) αφού βρίσκεται προς το -y, η δεξιά πλευρά το (1,0,0) αφού βρίσκεται προς το +x και η αριστερή το (-1,0,0) αφού βρίσκεται προς το -x.

Μετά ορίζουμε το VBO των normals **normalbuffer** και κάνουμε bind εκεί τον πίνακα **normals**.

```
GLuint normalbuffer;
glGenBuffers(1, &normalbuffer);
glBindBuffer(GL_ARRAY_BUFFER, normalbuffer);
glBufferData(GL_ARRAY_BUFFER, sizeof(normals), normals, GL_STATIC_DRAW);
```

Εικόνα 9.2: Ορισμός VBO για τα normals

Τέλος, αφού έχουμε ορίσει στο shader την υλοποίηση των normals στο attribute 3 πρέπει να το ενεργοποιήσουμε πριν κάνουμε draw με την μέθοδο **glVertexAttribPointer()** παίρνοντας τα στοιχεία ανά τρία.

Επομένως, στις γραμμές 3207 και 3222, δηλαδή για τον χαρακτήρα και τον θησαυρό αντίστοιχα, δίνουμε τις εντολές:

```
glEnableVertexAttribArray(3);
```

```
glBindBuffer(GL_ARRAY_BUFFER, normalbuffer);
```

```
glVertexAttribPointer(3, 3, GL_FLOAT, GL_FALSE, 0, (void*)0);
```

Παρόμοια εργαζόμαστε και για τον λαβύρινθο, όμως επειδή αποτελείται από 16 σχήματα, στην υλοποίηση μας, ορίζουμε τον πίνακα **maze_normals** για τα normals του, ο οποίος έχει τα ίδια στοιχεία με τον πίνακα **normals** επί 16 φορές ώστε να περαστούν σε κάθε σχήμα του λαβυρίνθου. Ο λαβύρινθος αποτελείται από ορθογώνια παραλληλόγραμμα διάφορων μεγεθών, αλλά αυτό δεν αλλάζει κάτι, αφού έχουν τα ίδια κάθετα διανύσματα με έναν κύβο.

Έτσι, όπως ακριβώς και πριν, στην γραμμή 3090 ορίζουμε το VBO **mazenormalbuffer** των normals του λαβυρίνθου και το κάνουμε bind πριν την σχεδίασή του στη γραμμή 3185.

Έχοντας τελειώσει με αυτό, θα αναλύσουμε το fragment shader και πως αυτό χρησιμοποιείται για το μοντέλο phong.

Πέρα από το fragment color και το UV που χρησιμεύουν για τους χαρακτήρες και για τον λαβύρινθο, θα πρέπει να εισάγουμε και άλλες τέσσερις μεταβλητές σαν in, οι οποίες αφορούν το φως και την κάμερα μας. Σαν out, παραμένει μόνο το color. Θα έχουμε και μια έξτρα μεταβλητή **LightPosition_worldspace** η οποία θα κρατάει την θέση του φωτός.

Μέσα στην main θα ορίσουμε ένα τρισδιάστατο διάνυσμα το οποίο θα κρατάει το χρώμα του φωτός αλλά και μια άλλη μεταβλητή (**LightPower**) η οποία ορίζει την ισχύ που θα έχει το φως.

```
#version 330 core

in vec4 fragmentColor;
in vec2 UV;

in vec3 Position_worldspace;
in vec3 Normal_cameraspace;
in vec3 EyeDirection_cameraspace;
in vec3 LightDirection_cameraspace;

out vec3 color;

uniform sampler2D myTextureSampler;
uniform bool useTexture; // True for textured objects, false for color-only objects
uniform mat4 MV;
uniform vec3 LightPosition_worldspace;

void main() {
    // Light properties
    vec3 LightColor = vec3(1.0, 1.0, 1.0);
    float LightPower = 50.0f;

    // Material properties
    vec3 MaterialDiffuseColor;
    if (useTexture) {
        // Use texture color for textured objects
        MaterialDiffuseColor = texture(myTextureSampler, UV).rgb;
    } else {
        // Use flat fragment color for non-textured objects
        MaterialDiffuseColor = fragmentColor.rgb;
    }
}
```

Εικόνα 9.3: fragment shader Μέρος 1

Έπειτα κάνουμε έναν έλεγχο για την τιμή του **useTexture** ώστε να γνωρίζουμε αν πρόκειται για τον θησαυρό(που απαιτεί texture) ή είτε για τον λαβύρινθο είτε για τον χαρακτήρα μας. Αν πρόκειται για τον θησαυρό τότε η μεταβλητή **MaterialDiffuseColor** θα παίρνει σαν τιμή το texture(), ενώ αν δεν πρόκειται για τον θησαυρό, θα παίρνει κανονικά το fragmentcolor.


```
vec3 MaterialAmbientColor = vec3(0.3, 0.3, 0.3) * MaterialDiffuseColor;
vec3 MaterialSpecularColor = vec3(0.3, 0.3, 0.3);

// Distance to light
float distance = length(LightPosition_worldspace - Position_worldspace);

// Normal and light direction
vec3 n = normalize(Normal_cameraspace);
vec3 l = normalize(LightDirection_cameraspace);

// Diffuse Lighting
float cosTheta = clamp(dot(n, l), 0, 1);

// Specular Lighting
vec3 E = normalize(EyeDirection_cameraspace);
vec3 R = reflect(-l, n);
float cosAlpha = clamp(dot(E, R), 0, 1);

// Combine lighting components
color =
    MaterialAmbientColor +
    MaterialDiffuseColor * LightColor * LightPower * cosTheta / (distance * distance) +
    MaterialSpecularColor * LightColor * LightPower * pow(cosAlpha, 5.0) / (distance * distance);
}
```

Εικόνα 9.4: fragment shader Μέρος 2

Έπειτα υπολογίζουμε τις τιμές των **MaterialAmbientColor** και **MaterialSpecularColor** καθώς και την μεταβλητή *distance* ώστε να χρησιμοποιηθεί στο τελικό υπολογισμό του χρώματος. Αμέσως παρακάτω έχουμε δύο κανονικοποιημένα διανύσματα τα οποία χρησιμοποιούνται για τον υπολογισμό της μεταβλητής **cosTheta**. Θα υπολογίσουμε επίσης και την τιμή της μεταβλητής **cosAlpha** και έπειτα θα υπολογίσουμε το τελικό χρώμα ανάλογα με τις τιμές των μεταβλητών που προέκυψαν παραπάνω. Η διαδικασία αυτή είναι σχεδόν ίδια με αυτή που εξηγήθηκε στο εργαστήριο.

Αναφορά φυσικά θα κάνουμε και για τον vertexShader καθώς χρησιμοποιείται και αυτός για τον υπολογισμό των θέσεων του φωτός και των κατευθύνσεων, ώστε να γίνουν σωστά οι τελικοί υπολογισμοί στο fragment Shader.

```
#version 330 core

layout(location = 0) in vec3 vertexPosition_modelspace;
layout(location = 1) in vec4 vertexColor;
//uv
layout(location = 2) in vec2 vertexUV;
// light
layout(location = 3) in vec3 vertexNormal_modelspace;

out vec4 fragmentColor;
//uv
out vec2 UV;

out vec3 Position_worldspace;
out vec3 Normal_cameraspace;
out vec3 EyeDirection_cameraspace;
out vec3 LightDirection_cameraspace;

uniform mat4 MVP;
uniform mat4 M;
uniform mat4 V;
uniform vec3 LightPosition_worldspace;

void main(){
    gl_Position = MVP * vec4(vertexPosition_modelspace,1);
    fragmentColor = vertexColor;
    //uv
    UV = vertexUV;
    Position_worldspace = (M * vec4(vertexPosition_modelspace,1)).xyz;

    vec3 vertexPosition_cameraspace = (V * M * vec4(vertexPosition_modelspace,1)).xyz;
    EyeDirection_cameraspace = vec3(0,0,0) - vertexPosition_cameraspace;

    vec3 LightPosition_cameraspace = (V * M * vec4(LightPosition_worldspace,1)).xyz;
    LightDirection_cameraspace = LightPosition_cameraspace - EyeDirection_cameraspace;

    Normal_cameraspace = (V * M * vec4(vertexNormal_modelspace,0)).xyz;
}
```

Εικόνα 9.5: Vertex Shader

10 ΕΡΩΤΗΜΑ BONUS: Δ) ΜΕΤΑΚΙΝΗΣΗ ΦΩΤΕΙΝΗΣ ΠΗΓΗΣ

Η μετακίνηση της φωτεινής πηγής γίνεται με την συνάρτηση **light_movement_function**. Αφού έχουμε ορίσει το διάνυσμα **lightPos** για τη θέση της πηγής, για την κίνησή του αρκεί να προσθέσουμε ή να αφαιρέσουμε την ταχύτητα στο x,y ή z, ανάλογα το είδος της μετακίνησης. Ορίσαμε την ταχύτητα ως 0.1 και αυτό προσθέτουμε ή αφαιρούμε κάθε φορά.

Χρησιμοποιούμε τους αριθμούς του numerical keypad ως input. Με το 8 το φως κινείται ευθεία προς τα πάνω δηλαδή παράλληλα στον άξονα z, με το 2 κινείται το ίδιο προς τα κάτω. Με το πλήκτρο 4 κινείται αριστερά παράλληλα στον άξονα x και με το 6 δεξιά. Με το πλήκτρο 7 κινείται μπροστά προς τον λαβύρινθο δηλαδή παράλληλα στον άξονα y και με το 1 απομακρύνεται.

```
659 void light_movement_function()
660 {
661     float lightSpeed = 0.1f; // Speed of light movement
662
663     if (glfwGetKey(window, GLFW_KEY_KP_8) == GLFW_PRESS) { // Move light up
664         lightPos.y += lightSpeed;
665     }
666     else if (glfwGetKey(window, GLFW_KEY_KP_2) == GLFW_PRESS) { // Move light down
667         lightPos.y -= lightSpeed;
668     }
669     else if (glfwGetKey(window, GLFW_KEY_KP_4) == GLFW_PRESS) { // Move light left
670         lightPos.x -= lightSpeed;
671     }
672     else if (glfwGetKey(window, GLFW_KEY_KP_6) == GLFW_PRESS) { // Move light right
673         lightPos.x += lightSpeed;
674     }
675     else if (glfwGetKey(window, GLFW_KEY_KP_7) == GLFW_PRESS) { // Move light forward
676         lightPos.z += lightSpeed;
677     }
678     else if (glfwGetKey(window, GLFW_KEY_KP_1) == GLFW_PRESS) { // Move light backward
679         lightPos.z -= lightSpeed;
680     }
681 }
```

Εικόνα 10.1: Η συνάρτηση **light_movement_function**

11 ΠΛΗΡΟΦΟΡΙΕΣ ΣΧΕΤΙΚΑ ΜΕ ΤΗΝ ΥΛΟΠΟΙΗΣΗ

Κατά την διάρκεια της άσκησης εργαστήκαμε με τα ακόλουθα:

Υπολογιστής 1

Λειτουργικό Σύστημα	Windows 10
Περιβάλλον/editor	Visual Studio 2022 (x64)

Υπολογιστής 2

Λειτουργικό Σύστημα	Windows 11
Περιβάλλον/editor	Visual Studio 2022 (x64)

Προκειμένου να λειτουργήσουν όλα, έχουμε κάνει include τις βιβλιοθήκες **windows.h** και **stb_image.h**. Για να αναγνωρίσει το πρόγραμμα την **stb_image** την βάλαμε σε ένα φάκελο στο working directory του project και τον προσθέσαμε στα additional include directories στα properties. Για την υλοποίηση του ηχητικού εφέ και την χρήση της συνάρτησης **PlaySound()** χρησιμοποιήσαμε την βιβλιοθήκη των windows **winmm.lib**, η οποία υπάρχει εγκατεστημένη στο λειτουργικό των windows. Την προσθέσαμε στα properties του project στο Linker->Input->Additional Dependencies.

12 ΑΞΙΟΛΟΓΗΣΗ ΤΗΣ ΛΕΙΤΟΥΡΓΙΑΣ ΤΗΣ ΟΜΑΔΑΣ

Κατά την διάρκεια της εργασίας η συνεργασία μας ήταν άριστη. Πριν ξεκινήσουμε οποιοδήποτε ερώτημα, συζητάγαμε πως θα μπορούσε να υλοποιηθεί το καθένα, αφού αυτό είναι ουσιαστικά σχεδόν πάντα η κύρια δυσκολία. Αφού καταλήγαμε κάπου, ξεκινούσε η υλοποίηση (το προγραμματιστικό κομμάτι δηλαδή).

Το τελευταίο αυτό κομμάτι, προσπαθήσαμε όσο γίνεται να το ισομοιάσουμε. Η συγγραφή της αναφοράς έγινε με όμοιο τρόπο.

13 ΠΗΓΕΣ

Παρακάτω αναφέρουμε κάποιες από τις πηγές τις οποίες συμβουλευτήκαμε κατά την διάρκεια της άσκησης, πέρα από τις οδηγίες που δόθηκαν στο εργαστήριο.

1. <https://www.opengl-tutorial.org/beginners-tutorials/tutorial-5-a-textured-cube/>
2. <https://www.opengl-tutorial.org/beginners-tutorials/tutorial-8-basic-shading/>
3. <https://stackoverflow.com/questions/8804880/use-playsound-in-c-opengl-to-play-sound-in-background>
4. <https://www.digitalocean.com/community/tutorials/random-number-generator-c-plus-plus>